



*Project Report On*

**Automated Proposal Generation based on  
Request for Proposal**

*Submitted in partial fulfillment of the requirements for the  
award of the degree of*

**Bachelor of Technology**

**in**

***Computer Science and Engineering***

**By**

**Nevin John George (U2203154)**

**Niranjana N (U2203163)**

**P A Abidh Muhammed (U2203171)**

**Rahul Rathishkumar Menon (U2203173)**

**Under the guidance of**

**Ms. Sangeetha Jamal**

**Department of Computer Science and Engineering  
Rajagiri School of Engineering & Technology (Autonomous)**

**(Parent University: APJ Abdul Kalam Technological University)**

**Rajagiri Valley, Kakkanad, Kochi, 682039**

**March 2026**

# CERTIFICATE

*This is to certify that the project report entitled "**Automated Proposal Generation based on Request for Proposal**" is a bonafide record of the work done by **Nevin John George (U2203154), Niranjana N (U2203163), P A Abidh Muhammed (U2203171) and Rahul Rathishkumar Menon (U2203173)**, submitted to the Rajagiri School of Engineering & Technology (RSET) (Autonomous) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in "Computer Science and Engineering" during the academic year 2025-2026.*

**Project Guide**

Ms. Sangeetha Jamal  
Assistant Professor  
Dept. of CSE  
RSET

**Project Co-ordinator**

Ms. Jisha Mary Jose  
Assistant Professor  
Dept. of CSE  
RSET

**Head of Department**

Dr. Preetha K G  
Professor  
Dept. of CSE  
RSET

# ACKNOWLEDGMENT

We wish to express our sincere gratitude towards **Rev. Dr. Jaison Paul Mulerikkal** CMI, Principal, RSET, and **Dr. Preetha K.G.**, Head of the Department, Department of Computer Science and Engineering for providing us with the opportunity to undertake our project, "Automated Proposal Generation based on Request for Proposal".

We are highly indebted to our project co-ordinator, **Ms. Jisha Mary Jose**, Assistant Professor, Dept. of CSE.

It is indeed our pleasure and a moment of satisfaction for us to express our sincere gratitude to our project guide **Ms. Sangeetha Jamal**, Assistant Professor, Dept. of CSE for her patience and all the priceless advice and wisdom she has shared with us.

We would also like to sincerely acknowledge **Mr. Samil Kumar**, Director-Service Delivery, 10xDS – Exponential Digital Solutions, Infopark, Kochi, Kerala, India, for his continuous guidance and valuable support throughout this work. His suggestions, technical expertise, and constructive feedback have played a significant role in overcoming the challenges and improving the results of this study.

Last but not the least, We would like to express our sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

**Nevin John George**

**Niranjana N**

**P A Abidh Muhammed**

**Rahul Rathishkumar Menon**

# Abstract

Organizations frequently compete for business by responding to Tenders and Requests for Proposals (RFPs), a traditionally complex and time-consuming process that demands extensive collaboration, detailed analysis, and strict compliance with client requirements. This project leverages Generative AI to streamline proposal creation by automating key stages such as RFP analysis, competency evaluation, and draft proposal generation. The system addresses common challenges including inefficiencies in manual RFP analysis, inconsistent proposal quality, and suboptimal use of expert resources. Key features include automated extraction and summarization of tender requirements using Natural Language Processing, semantic competency matching between organizational strengths and RFP needs using the all-MiniLM-L6-v2 sentence transformer, and AI-driven proposal generation using a Retrieval Augmented Generation (RAG) pipeline. In this pipeline, relevant RFP sections are retrieved via vector embeddings and passed as context to LLaMA 3.2, an open-source large language model running locally via Ollama, generating structured sections including executive summary, technical approach, timeline, and compliance checklist. The system additionally integrates the Gemini API as a parallel generation approach, enabling a comparative study of local versus cloud-based generation in terms of output quality, cost, and privacy. By automating repetitive tasks and ensuring tailored, high-quality outputs, the system empowers teams to focus on strategic messaging and creativity, accelerating response times and increasing success rates in competitive bidding environments.

# Contents

|                                                                                                                                                                                       |            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <b>Acknowledgment</b>                                                                                                                                                                 | <b>i</b>   |
| <b>Abstract</b>                                                                                                                                                                       | <b>ii</b>  |
| <b>List of Abbreviations</b>                                                                                                                                                          | <b>vii</b> |
| <b>List of Figures</b>                                                                                                                                                                | <b>ix</b>  |
| <b>List of Tables</b>                                                                                                                                                                 | <b>x</b>   |
| <b>1 Introduction</b>                                                                                                                                                                 | <b>1</b>   |
| 1.1 Background . . . . .                                                                                                                                                              | 1          |
| 1.2 Problem Statement . . . . .                                                                                                                                                       | 1          |
| 1.3 Scope and Motivation . . . . .                                                                                                                                                    | 2          |
| 1.4 Objectives . . . . .                                                                                                                                                              | 2          |
| 1.5 Risks & Challenges . . . . .                                                                                                                                                      | 3          |
| 1.6 Assumptions . . . . .                                                                                                                                                             | 3          |
| 1.7 Societal / Industrial Relevance . . . . .                                                                                                                                         | 4          |
| 1.8 Organization of the Report . . . . .                                                                                                                                              | 4          |
| 1.9 Summary of this chapter . . . . .                                                                                                                                                 | 5          |
| <b>2 Literature Survey</b>                                                                                                                                                            | <b>6</b>   |
| 2.1 Introduction . . . . .                                                                                                                                                            | 6          |
| 2.2 Survey of Research Papers . . . . .                                                                                                                                               | 6          |
| 2.2.1 Financial Report Chunking for Effective Retrieval Augmented Gen-<br>eration, Antonio Jimeno Yepes, Yao You, Jan Milczek, Sebastian<br>Laverde, and Leah Li, 2024, [1] . . . . . | 6          |

|          |                                                                                                                                                                                                                                                    |           |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| 2.2.2    | Unlocking the Potential of Competency Exam Data with Machine Learning: Improving Higher Education Evaluation, Ala Smadi and Ahmad Al-Qerem and Ahmad Nabot and Issam Jebreen and Amjad Aldweesh and Mohammad Alauthman et al., 2023, [2] . . . . . | 9         |
| 2.2.3    | Questioning the Validity of Summarization Datasets and Improving Their Factual Consistency, Yanzhu Guo, Chloé Clavel, Moussa Kamal Eddine, and Michalis Vazirgiannis, 2022, [3] . . . . .                                                          | 12        |
| 2.2.4    | An Industrial Experience Report on Model-based, AI-enabled Proposal Development for an RFP/RFI, P. Nistala et al., 2024, [4] . . .                                                                                                                 | 15        |
| 2.3      | Comparison and Gaps Identified . . . . .                                                                                                                                                                                                           | 18        |
| 2.3.1    | Overview of Surveyed Approaches . . . . .                                                                                                                                                                                                          | 18        |
| 2.3.2    | Identified Research Gaps . . . . .                                                                                                                                                                                                                 | 20        |
| 2.4      | Summary of the Chapter . . . . .                                                                                                                                                                                                                   | 20        |
| <b>3</b> | <b>System Design</b>                                                                                                                                                                                                                               | <b>21</b> |
| 3.1      | Overall System Architecture . . . . .                                                                                                                                                                                                              | 21        |
| 3.2      | Design Diagrams . . . . .                                                                                                                                                                                                                          | 22        |
| 3.2.1    | Use Case Diagram . . . . .                                                                                                                                                                                                                         | 23        |
| 3.2.2    | Sequence Diagram . . . . .                                                                                                                                                                                                                         | 25        |
| 3.2.3    | Entity-Relationship Diagram . . . . .                                                                                                                                                                                                              | 28        |
| 3.3      | Component Design . . . . .                                                                                                                                                                                                                         | 28        |
| 3.4      | Algorithms Used and Design . . . . .                                                                                                                                                                                                               | 29        |
| 3.4.1    | KeyBERT Algorithm . . . . .                                                                                                                                                                                                                        | 29        |
| 3.4.2    | Document Parsing Algorithms . . . . .                                                                                                                                                                                                              | 30        |
| 3.4.3    | Retrieval Augmented Generation (RAG) . . . . .                                                                                                                                                                                                     | 30        |
| 3.4.4    | LLaMA 3.2 via Ollama . . . . .                                                                                                                                                                                                                     | 31        |
| 3.4.5    | System Architecture Design . . . . .                                                                                                                                                                                                               | 32        |
| 3.4.6    | Design Patterns Implemented . . . . .                                                                                                                                                                                                              | 33        |
| 3.5      | Software and Hardware Requirements . . . . .                                                                                                                                                                                                       | 33        |
| 3.5.1    | Software Requirements . . . . .                                                                                                                                                                                                                    | 33        |
| 3.5.2    | Hardware Requirements . . . . .                                                                                                                                                                                                                    | 35        |
| 3.6      | Dataset Identified . . . . .                                                                                                                                                                                                                       | 35        |

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| 3.7      | Database . . . . .                                  | 36        |
| 3.7.1    | Database Management System . . . . .                | 36        |
| 3.7.2    | Database Schema . . . . .                           | 36        |
| 3.7.3    | Database Indexing Strategy . . . . .                | 37        |
| 3.8      | Budget . . . . .                                    | 37        |
| 3.8.1    | Cost Analysis . . . . .                             | 37        |
| 3.8.2    | Cost-Benefit Analysis . . . . .                     | 38        |
| 3.9      | Module Division and Work Responsibilities . . . . . | 39        |
| 3.9.1    | Module Breakdown . . . . .                          | 39        |
| 3.9.2    | Work Distribution . . . . .                         | 44        |
| 3.10     | Key Deliverables – Expected Outputs . . . . .       | 44        |
| 3.10.1   | Software Deliverables . . . . .                     | 44        |
| 3.10.2   | Expected Performance Metrics . . . . .              | 47        |
| 3.10.3   | Output Formats . . . . .                            | 47        |
| 3.11     | Project Timeline . . . . .                          | 48        |
| 3.12     | UI Design . . . . .                                 | 48        |
| 3.12.1   | Design Principles . . . . .                         | 48        |
| 3.12.2   | UI Components . . . . .                             | 49        |
| 3.13     | Summary of Chapter . . . . .                        | 50        |
| <b>4</b> | <b>Results</b>                                      | <b>51</b> |
| 4.1      | Introduction . . . . .                              | 51        |
| 4.2      | Document Upload and Processing . . . . .            | 51        |
| 4.3      | Keyword Extraction Results . . . . .                | 52        |
| 4.4      | Document Summary Results . . . . .                  | 54        |
| 4.5      | RFP Metadata Extraction Results . . . . .           | 56        |
| 4.6      | RFP Evaluation Results . . . . .                    | 56        |
| 4.7      | Generate Proposal Banner . . . . .                  | 58        |
| 4.8      | Proposal Generation Results . . . . .               | 59        |
| 4.9      | Summary of Chapter . . . . .                        | 65        |
| <b>5</b> | <b>Conclusion</b>                                   | <b>66</b> |
| 5.1      | System Implementation Overview . . . . .            | 66        |

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| 5.2 Summary of Chapter . . . . .                                           | 67        |
| <b>List of Publications</b>                                                | <b>68</b> |
| <b>References</b>                                                          | <b>69</b> |
| <b>Appendices</b>                                                          | <b>70</b> |
| <b>Appendix A: Presentation</b>                                            | <b>71</b> |
| <b>Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes</b> | <b>93</b> |
| <b>Appendix C: CO-PO-PSO Mapping</b>                                       | <b>97</b> |



## List of Abbreviations

|      |   |                                                               |
|------|---|---------------------------------------------------------------|
| AI   | – | Artificial Intelligence                                       |
| API  | – | Application Programming Interface                             |
| DB   | – | Database                                                      |
| DL   | – | Deep Learning                                                 |
| DOCX | – | Microsoft Word Document Format                                |
| GPT  | – | Generative Pre-trained Transformer                            |
| HTML | – | HyperText Markup Language                                     |
| JSON | – | JavaScript Object Notation                                    |
| LLM  | – | Large Language Model                                          |
| ML   | – | Machine Learning                                              |
| NLP  | – | Natural Language Processing                                   |
| PDF  | – | Portable Document Format                                      |
| RAG  | – | Retrieval-Augmented Generation                                |
| RFP  | – | Request for Proposal                                          |
| RFx  | – | Request for Information / Proposal / Quotation (Generic Term) |
| UI   | – | User Interface                                                |
| UX   | – | User Experience                                               |
| AUC  | – | Area Under the Curve                                          |

## List of Figures

|     |                                                                                                                                                                                                                |    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Competency Exam Data Processing Pipeline Architecture (illustrates the workflow from raw exam data through preprocessing, segmentation, training/test splits, and classification model evaluation)[2]. . . . . | 10 |
| 2.2 | Filtered Summarization Dataset Architecture (shows how CNN/DM, XSUM, XLSUM are filtered using BERTScore, BARTScore, and DAE to create SummFC for fine-tuning BART models)[3]. . . . .                          | 13 |
| 2.3 | Natural Language Document Interpretation Framework (depicts the pattern interpreter modules and their interaction with the modeling framework layers to convert NL text into formal models)[4]. . . . .        | 17 |
| 3.1 | Architecture of AI-based RFP Proposal Generation System, showing user input, preprocessing, compliance checking, scoring, and automated proposal generation/rejection flow. . . . .                            | 22 |
| 3.2 | Use Case Diagram. . . . .                                                                                                                                                                                      | 23 |
| 3.3 | Sequence Diagram. . . . .                                                                                                                                                                                      | 25 |
| 3.4 | Entity-Relationship Diagram. . . . .                                                                                                                                                                           | 28 |
| 3.5 | Project Gantt Chart. . . . .                                                                                                                                                                                   | 48 |
| 4.1 | Document Upload Interface showing file selection and Upload and Evaluate button. . . . .                                                                                                                       | 52 |
| 4.2 | Extracted Keywords Display showing the top 15 semantically ranked keywords with relevance scores. . . . .                                                                                                      | 53 |
| 4.3 | Document Summary panel showing the structured summary. . . . .                                                                                                                                                 | 55 |
| 4.4 | RFP Evaluation Results panel displaying animated progress bars for technical, budget, and timeline fit scores alongside the overall fit score and decision badge. . . . .                                      | 57 |
| 4.5 | Upload Interface with ACCEPT Decision and Generate Proposal Banner confirming bid viability. . . . .                                                                                                           | 59 |

|      |                                                                                                                             |    |
|------|-----------------------------------------------------------------------------------------------------------------------------|----|
| 4.6  | Proposal Generation Page showing the animated loading screen with sequential progress steps during LLaMA inference. . . . . | 60 |
| 4.7  | Generated Proposal Panel-a, showing Executive Summary, download, print and regenerate button. . . . .                       | 61 |
| 4.8  | Generated Proposal Panel-b, showing Technical Approach, Timeline & Milestones, and Compliance Checklist sections. . . . .   | 61 |
| 4.9  | Generated Proposal Panel-c, showing Technical Approach section. . . . .                                                     | 62 |
| 4.10 | Generated Proposal Panel-d, showing Timeline & Milestones section. . . . .                                                  | 63 |
| 4.11 | Generated Proposal Panel-e, showing Compliance Checklist section. . . . .                                                   | 64 |

## List of Tables

|     |                                                                                                    |    |
|-----|----------------------------------------------------------------------------------------------------|----|
| 2.1 | Comparative Analysis of Surveyed Research Papers . . . . .                                         | 19 |
| 3.1 | Document Table Schema. Stores uploaded document metadata and content preview. . . . .              | 36 |
| 3.2 | Keyword Table Schema. Stores unique keywords extracted across all documents. . . . .               | 37 |
| 3.3 | DocumentKeyword Junction Table Schema. Links documents with keywords and relevance scores. . . . . | 37 |
| 3.4 | Development Phase Budget . . . . .                                                                 | 38 |
| 3.5 | Deployment Phase Budget Estimate . . . . .                                                         | 38 |
| 3.6 | Work Distribution Among Team Members . . . . .                                                     | 44 |
| 3.7 | Expected Performance Metrics . . . . .                                                             | 47 |
| 1   | CO-PO and CO-PSO Mapping . . . . .                                                                 | 98 |

# Chapter 1

## Introduction

### 1.1 Background

In modern business environments, responding to Requests for Proposals (RFPs) has become a critical aspect of acquiring new clients and sustaining competitiveness. Companies spend substantial time and resources analyzing lengthy tender documents, identifying compliance requirements, and preparing customized proposals. These manual processes often result in inefficiencies, inconsistencies, and missed opportunities. The emergence of Artificial Intelligence (AI), particularly Generative AI (GenAI), has introduced new possibilities for automating knowledge-intensive tasks such as proposal writing. By integrating Natural Language Processing (NLP), machine learning, and knowledge modeling, proposal generation can now be optimized to deliver faster, more accurate, and tailored responses that align with client requirements.

### 1.2 Problem Statement

Traditional proposal development is a labor-intensive process involving multiple teams working across different domains. This often leads to duplicated effort, human errors, and inconsistent documentation quality. Additionally, understanding complex RFPs requires domain expertise and significant time investment. Many organizations face challenges in ensuring compliance with specific tender criteria and maintaining a repository of reusable knowledge for future proposals [5]. Therefore, there is a need for an intelligent, AI-powered solution that can automatically interpret RFPs, match organizational capabilities, and generate structured proposal drafts, ensuring efficiency, accuracy, and standardization.

### 1.3 Scope and Motivation

- **Scope**

The project focuses on designing and implementing a Generative AI-based Proposal Creation System that automates critical stages of the proposal process. The system accepts RFPs in multiple formats, analyzes content to extract key requirements, evaluates organizational readiness and compliance, and generates proposal drafts using predefined templates and knowledge models. It integrates modules such as keyword extraction, scoring metrics, proposal validation, and learning engines to ensure accuracy and adaptability. The solution also prioritizes data security, maintainability, and scalability, making it suitable for industrial adoption across various business sectors.

- **Motivation**

Organizations often spend substantial time and resources responding to RFPs, resulting in delayed submissions and inconsistent proposal quality. Manual handling of large, complex documents increases the likelihood of oversight and errors, leading to missed business opportunities. With the rise of AI-driven automation, there is a strong need for intelligent systems that can accelerate RFP responses without compromising quality or compliance. This project is motivated by the potential of Generative AI to transform traditional proposal management into a faster, smarter, and more reliable process [6]. By automating repetitive tasks and learning from past submissions, the system empowers proposal teams to focus on strategic inputs and innovation rather than routine documentation.

### 1.4 Objectives

- To design and develop an AI-powered proposal creation system that automates RFP analysis and proposal generation.
- To evaluate organizational competency based on resource availability and project requirements.

- To generate accurate, compliant, and professionally formatted proposal drafts tailored to specific clients.
- To employ NLP and GenAI models for extracting and summarizing key RFP details and evaluation criteria.
- To integrate a scoring mechanism for measuring compliance and relevance based on extracted keywords.
- To enhance system performance through modular design and continuous learning from user feedback.
- To ensure data confidentiality and integrity during document processing and storage.

## 1.5 Risks & Challenges

Developing an AI-powered automated proposal generation system introduces several challenges:

- **Data Quality:** Inconsistent or incomplete RFP data can impact system performance and result in inaccurate outputs.
- **Contextual Understanding:** Misinterpretation of client requirements may lead to incorrect proposal responses.
- **Security and Confidentiality:** Sensitive data in RFPs must be securely processed to prevent information leaks.
- **Integration:** Seamless integration with existing enterprise knowledge systems and databases can be complex.
- **Model Maintenance:** Continuous model retraining and knowledge updates are essential to sustain relevance and accuracy.

## 1.6 Assumptions

- All input RFP documents are provided in supported digital formats (PDF, DOCX, or text).

- The AI model is trained using diverse and representative datasets of past RFPs and proposals.
- Domain experts are available to review AI-generated proposals for quality assurance.
- The system will operate within secure organizational environments compliant with data protection norms.

## 1.7 Societal / Industrial Relevance

This project holds significant industrial relevance, as it aligns with modern business transformation efforts driven by AI and automation. The system improves operational efficiency by reducing proposal turnaround time from days to hours, allowing companies to participate in more tenders with consistent quality. It enhances decision-making, ensures compliance, and minimizes manual workload in presales departments. On a societal level, such automation promotes digital innovation, boosts organizational productivity, and contributes to economic growth by enabling faster business collaboration and transparent bidding processes.

## 1.8 Organization of the Report

This report is structured as follows.

- **Chapter 1 – Introduction:** Provides an overview of the project background, motivation, objectives, problem definition, challenges, and industrial relevance.
- **Chapter 2 – Literature Survey:** Reviews related research papers and identifies existing gaps in AI-based proposal automation.
- **Chapter 3 – System Design:** Describes the architecture, design diagrams, modules, algorithms, datasets, and expected outputs.
- **Chapter 4 – Results:** Presents the outputs of each functional module through annotated interface screenshots, demonstrating keyword extraction, document summarization, RFP evaluation scoring, and AI-generated proposal sections produced by the system.



- **Chapter 5 – Conclusion:** Summarises the key accomplishments of the implemented system, highlights the contributions of each module, and outlines directions for future enhancement and production deployment.

## 1.9 Summary of this chapter

This chapter introduced the concept and need for an AI-based automated proposal generation system to revolutionize how organizations respond to RFPs. It outlined the motivation behind using Generative AI and NLP to streamline proposal creation, enhance compliance, and reduce manual workload. The objectives of the system, such as automating RFP analysis, competency evaluation, and dynamic proposal generation, were clearly defined. The challenges related to data quality, compliance, and system integration were highlighted, along with assumptions and industrial benefits. Finally, the chapter established the overall organization of the report, setting the foundation for the subsequent chapters that delve deeper into system design, implementation, and evaluation.

## Chapter 2

### Literature Survey

#### 2.1 Introduction

The field of automated proposal generation has evolved significantly with the rise of Natural Language Processing (NLP) and Generative Artificial Intelligence (GenAI). Traditionally, proposal writing was a manual process requiring extensive human effort to analyze RFPs, align company capabilities, and prepare compliant documents. Recent advances in AI, particularly large language models (LLMs), have enabled automation in document understanding, summarization, and content generation. This chapter explores research and existing systems that leverage AI for document automation, RFP analysis, and proposal generation, identifying the gaps that the proposed system aims to address.

#### 2.2 Survey of Research Papers

##### 2.2.1 Financial Report Chunking for Effective Retrieval Augmented Generation, Antonio Jimeno Yepes, Yao You, Jan Milczek, Sebastian Laverde, and Leah Li, 2024, [1]

##### Overview

This paper addresses the critical challenge of document chunking in Retrieval Augmented Generation (RAG) systems, specifically for financial reporting documents. The study proposes an element-based chunking strategy that leverages document structure rather than arbitrary token limits, demonstrating superior performance in question-answering tasks on financial documents.

## Methodology

### 1. Dataset and Data Source

The study utilized the FinanceBench dataset consisting of 150 question-answer instances from 84 unique SEC financial reports (10-Ks, 10-Qs, and 8-Ks). The researchers successfully recovered 80 documents containing 141 questions from the original 150. The documents varied significantly in length, spanning from 4 pages to 549 pages with an average of 147.34 pages (standard deviation 97.78), totaling 11,787 pages combined. Using the OpenAI tokenizer (cl100k\_base), documents averaged 102,444.35 tokens with a standard deviation of 61,979.45 tokens, demonstrating substantial variability in document lengths.

### 2. Preprocessing

Documents were processed using the Unstructured open-source library with the Chipper vision encoder-decoder model. The Chipper model, inspired by Donut, outputs JSON representations of documents with element types, bounding boxes, and corresponding text. The preprocessing identified multiple element types including NarrativeText (61,780 entities), Title (29,664), ListItem (33,054), UncategorizedText (9,400), Table (7,700), Header (3,959), Footer (1,026), and others, totaling 146,921 elements. For chunk generation, elements smaller than 2,048 characters were merged iteratively with following elements. New chunks were initiated when title or table elements were encountered, preserving table integrity. Three types of metadata were generated: up to 6 representative keywords via GPT-4, summarized paragraphs, and naive representations using first two sentences or table descriptions.

### 3. Model Architecture

The RAG system comprised three main components: Indexing and Retrieval using Weaviate vector database with a sentence transformer encoder trained on 256M+ question-answer pairs, implementing approximate nearest neighbor algorithms for fast retrieval, Generation using GPT-4 as the large language model with a specifically designed prompt conditioning answers on query and retrieved chunks, and Chunking strategies including baseline token-based chunking (128, 256, 512 tokens) and element-based chunking lever-

aging document structure. The element-based method created chunks averaging 260.57 per document with 96.20 tables preserved.

#### **4. Evaluation Metrics**

The study employed multiple evaluation metrics: Page-level retrieval accuracy measuring if correct page was retrieved, Paragraph-level retrieval using ROUGE and BLEU scores comparing retrieved contexts to ground truth evidence, and Question-Answering accuracy using both manual evaluation and automated GPT-4-based comparison. Cross-validation was performed to ensure statistical significance of results using the Wilcoxon signed-rank test.

#### **Results**

Element-based chunking (Chipper) achieved the best Q&A accuracy of 53.19% (manual evaluation) compared to Base 512’s 48.23% and Base 256’s 36.88%. The Chipper Aggregation method achieved the highest page accuracy at 84.40% and best paragraph-level scores (ROUGE: 0.568, BLEU: 0.452). Notably, element-based chunking required only 20,843 chunks compared to 64,058 for Base 128, offering 50% reduction in indexing costs while maintaining superior performance. The method showed consistent results across different metrics, with Prefix & Table Description Chipper achieving 67.38% page accuracy and 20.57% "no answer" rate. The study demonstrated that basic 512-token chunking performed closest to element-based approaches but failed to maintain coherent context, resulting in lower ROUGE and BLEU scores.

#### **Conclusion**

The paper demonstrates that element-based chunking significantly outperforms token-based strategies for RAG systems in financial documents. Key contributions include a generalizable chunking approach eliminating hyperparameter tuning, improved Q&A performance (50% to 53.19%), and 50% reduction in indexing costs. This research provides validated approaches for document chunking and RAG implementation relevant to business document contexts like RFP systems.

**2.2.2 Unlocking the Potential of Competency Exam Data with Machine Learning: Improving Higher Education Evaluation, Ala Smadi and Ahmad Al-Qerem and Ahmad Nabot and Issam Jebreen and Amjad Aldweesh and Mohammad Alauthman et al., 2023, [2]**

**Overview**

This paper presents a comprehensive framework for evaluating academic programs in Jordanian higher education using machine learning techniques applied to competency exam data. The study addresses the challenge of student performance prediction by creating segmented datasets based on student characteristics, demonstrating that targeted models outperform general models trained on complete datasets.

**Methodology**

**1. Dataset and Data Source**

The dataset comprised competency exam (CE) results from Jordanian universities, including 45,392 records in the complete dataset. The competency exam consists of two levels: macro level with 30 competencies evaluating broader skill sets, and micro level with 45 competencies specific to student majors. The dataset included demographic information (gender, university type), academic data (average university degree, program family), participation method, and exam outcomes (pass/fail) for both macro and micro competencies. Sub-datasets were created: macro exam (36,977 records) and micro exam (8,415 records). Each record contained student features including university quality indicators, grade levels, and contextual information about the examination process.

**2. Preprocessing**

Data segmentation was performed based on significant demographic and academic characteristics to address data heterogeneity. Feature selection was conducted using Decision Trees (DT), Random Forest (RF), and Extra Trees (ET) algorithms to identify the most important features. The most important features identified across datasets were: for all-dataset - student grade and university; for macro exam - grade, survey satisfaction, and university; for micro exam - participation method, grade, and university. The dataset

was split into training and test sets following standard machine learning practices. Data cleaning included handling missing values, normalizing features, and encoding categorical variables appropriately for each ML algorithm.

### 3. Model Architecture

Five classification algorithms were implemented and compared: Logistic Regression as a baseline linear model, K-Nearest Neighbors (KNN) Classifier implementing distance-based classification, Multi-Layer Perceptron (MLP) representing neural network approaches with multiple hidden layers, Support Vector Machines (SVM) using kernel methods for non-linear classification, and Decision Trees for rule-based classification. Each model was configured with appropriate hyperparameters and trained separately on the full dataset and segmented sub-datasets. Cross-validation techniques were employed to ensure model generalizability and prevent overfitting.

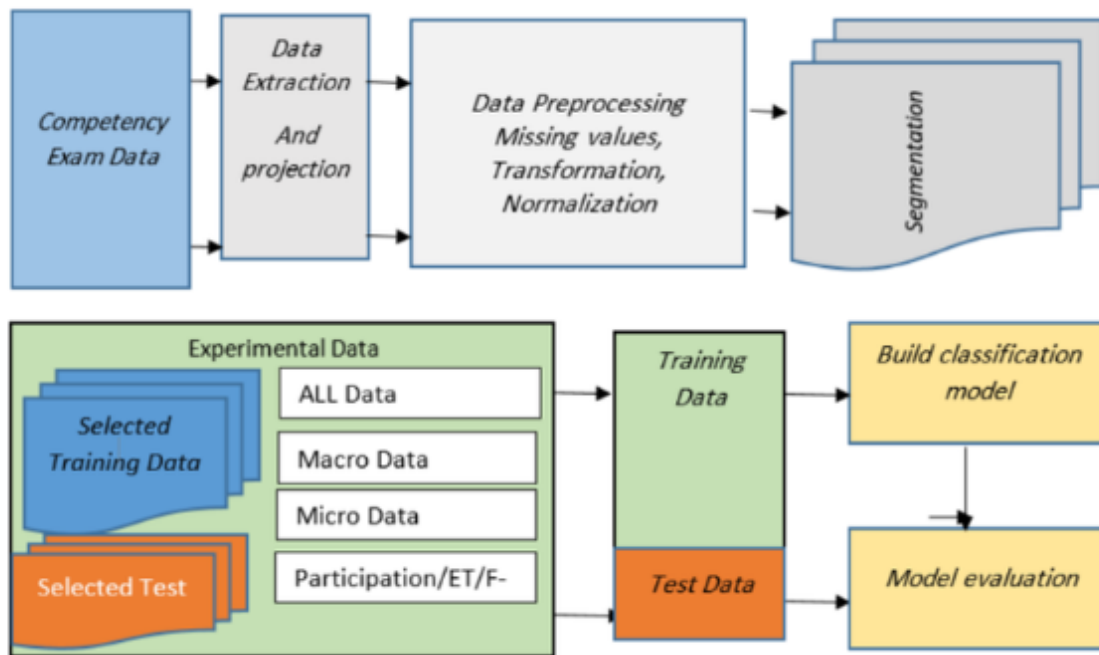


Figure 2.1: Competency Exam Data Processing Pipeline Architecture (illustrates the workflow from raw exam data through preprocessing, segmentation, training/test splits, and classification model evaluation)[2].

## 4. Evaluation Metrics

Model performance was evaluated using five key metrics: Accuracy measuring overall correct classification rate, F1-score providing harmonic mean of precision and recall, AUC (Area Under Curve) measuring model’s ability to distinguish between classes, Recall measuring the proportion of actual positives correctly identified, and Precision measuring the proportion of predicted positives that were correct. Statistical significance testing ( $p < 0.05$ ) was confirmed using the Wilcoxon signed-rank test to validate all results.

## Results

For the all-dataset case study (45,392 records), MLP achieved the best overall performance with accuracy of 0.651784, F1-score of 0.725801, AUC of 0.629099, recall of 0.826877, and precision of 0.646744. KNN Classifier performed second with 0.634087 accuracy. In the macro exam case study (36,977 records), MLP again achieved the highest accuracy of 0.624482 with F1-score 0.72014 and AUC 0.579259, though Logistic Regression showed the highest recall (0.973866) but lowest precision (0.607287). For the micro exam case study, KNN Classifier unexpectedly outperformed with accuracy of 0.753663, attributed to the strong connection between features and student specialization. The experiments confirmed that models trained on sub-datasets based on specific features (exam type, participation method, program family) outperformed the base model trained on the complete dataset. Feature importance analysis revealed that student grade consistently emerged as the most critical predictor across all models, followed by university quality and survey satisfaction in specific contexts.

## Conclusion

The study demonstrates that ML algorithms effectively recognize patterns in student performance for evaluating academic programs. Key contributions include a comprehensive evaluation framework, publicly available dataset, validation that data segmentation improves performance, and evidence that MLP models produce more accurate predictions for robust frameworks. This research demonstrates effective ML classification applicable to competency-based matching in RFP response systems.

### **2.2.3 Questioning the Validity of Summarization Datasets and Improving Their Factual Consistency, Yanzhu Guo, Chloé Clavel, Moussa Kamal Eddine, and Michalis Vazirgiannis, 2022, [3]**

#### **Overview**

This paper addresses critical issues in popular abstractive summarization benchmark datasets, particularly factual inconsistency problems. The authors combine state-of-the-art factual consistency models to identify and filter problematic samples, releasing SummFC, an improved dataset that produces better-performing summarization models across multiple quality dimensions.

#### **Methodology**

##### **1. Dataset and Data Source**

The study analyzed three major summarization benchmark datasets: CNN/DailyMail (CNN/DM) with 287,113 training samples, originally created for question answering then adapted for summarization by concatenating bullet-point highlights, XSUM with 204,045 training samples from BBC articles using first lines as summaries, and XL-SUM with 306,522 English training samples, also from BBC but using bold paragraphs instead of first lines. Documents in CNN/DM averaged 788 tokens with 54-token summaries, XSUM averaged 430 tokens with 23-token summaries, and XLSUM averaged 530 tokens with 24-token summaries. The FRANK benchmark was used for validation, containing 2,250 human-annotated model summaries (1,250 from CNN/DM, 1,000 from XSUM) with categorical error scores.

##### **2. Preprocessing**

Three state-of-the-art factual consistency models were applied to score all document-summary pairs: BERTScore\_Art computing token-level similarity using BERT embeddings with greedy matching (precision scores used), BARTScore formulating evaluation as text generation probability using encoder-decoder BART model, and DAE (Dependency Arc Entailment) decomposing summaries into dependency arcs with independent factuality judgments aggregated via mean scores. For each dataset, the bottom 25% of



samples scored by each factuality model were filtered out. The final SummFC dataset retained only the intersection of top 75% samples identified by all three models, ensuring comprehensive factuality coverage across different error types.

### 3. Model Architecture

The evaluation pipeline used pre-trained BART models fine-tuned separately on original datasets and SummFC selections. The BART architecture is an encoder-decoder transformer model pre-trained using denoising objectives. For text generation, BART employed beam search decoding. The factual consistency models operated as follows: BERTScore\_Art used contextual embeddings from pre-trained BERT, BARTScore computed conditional generation probabilities  $p(\text{summary}|\text{document})$  using pre-trained BART, and DAE trained a binary entailment classifier on human annotations with RoBERTa backbone, making token-level factuality judgments. All models output continuous scores enabling ranking and threshold-based filtering.

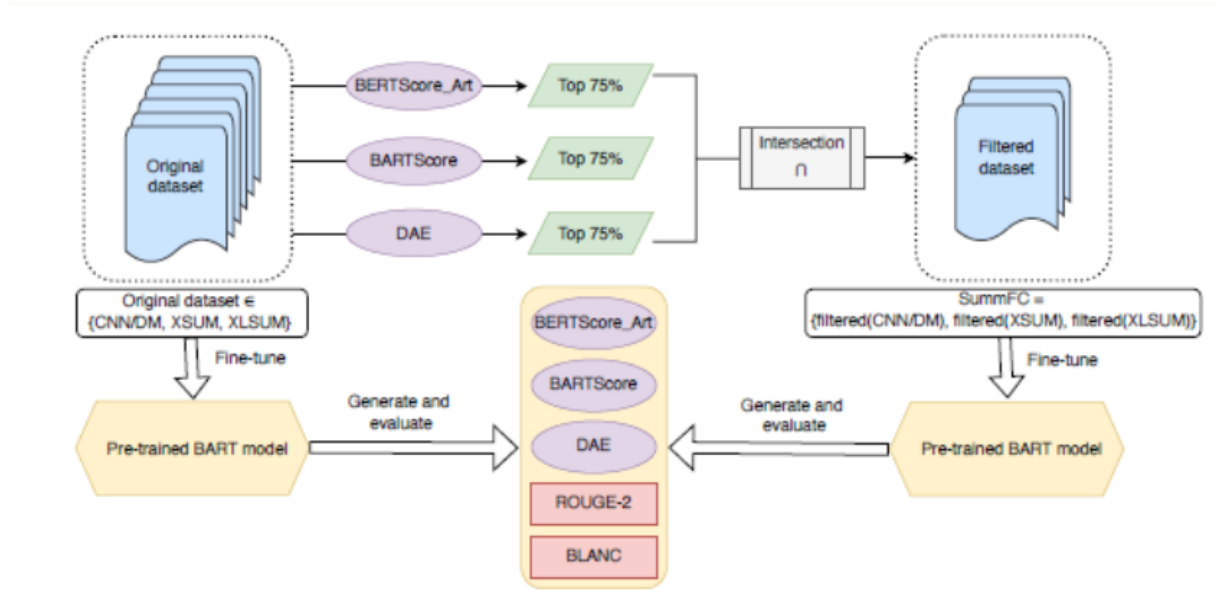


Figure 2.2: Filtered Summarization Dataset Architecture (shows how CNN/DM, XSUM, XLSUM are filtered using BERTScore, BARTScore, and DAE to create SummFC for fine-tuning BART models)[3].

#### 4. Evaluation Metrics

Multiple dimensions of summary quality were evaluated: Factual consistency using the three factuality models (BERTScore\_Art, BARTScore, DAE), BLANC (BLANC-help version) as reference-free metric measuring how well summaries help language models with masked token tasks, indicating informativeness, ROUGE-2 computing bigram overlap with reference summaries to measure saliency and coverage of important information. Statistical significance was confirmed using Wilcoxon signed-rank test ( $p < 0.05$ ). Evaluation was performed on test sets, with ROUGE-2 using filtered test sets and other metrics using unfiltered test sets since they are reference-free.

#### Results

The filtration methodology resulted in selection ratios of 53.52% for CNN/DM (153,666 samples), 58.03% for XSUM (118,427 samples), and 52.59% for XLSUM (161,200 samples). High overlap between models' filtered samples validated the combined approach. For CNN/DM, models trained on SummFC showed improvements: BERTScore\_Art increased from 0.9346 to 0.9364, DAE improved from 0.9001 to 0.9073, and BLANC increased from 0.1593 to 0.1556, while maintaining comparable ROUGE-2 scores (23.67 vs 23.77). For XSUM, SummFC training yielded: BERTScore\_Art 0.8982 (vs 0.8944 baseline), BARTScore -2.383 (vs -2.452), DAE 0.4473 (vs 0.4201), BLANC 0.07378 (vs 0.07316), and ROUGE-2 21.50 (vs 21.78). XLSUM showed the most dramatic improvements: DAE increased from 0.4695 to 0.5120, BLANC from 0.06886 to 0.06983, BERTScore\_Art from 0.8961 to 0.8989, and BARTScore from -2.594 to -2.501. Experiments comparing different threshold levels (25%, 52.59%, 75%, 100%) demonstrated that combined model filtration outperformed single-model selection at equivalent thresholds. The study confirmed that improved training data quality enhances not only factual consistency but also informativeness and saliency of generated summaries.

#### Conclusion

The paper provides evidence that popular summarization datasets contain significant factual inconsistency issues. Key contributions include systematic investigation of dataset quality, optimal filtration methodology combining three complementary models, release of

SummFC with improved factual consistency, and empirical proof that better training data improves model performance across multiple quality dimensions. This demonstrates the importance of training data quality for ensuring factual accuracy in generated documents.

#### **2.2.4 An Industrial Experience Report on Model-based, AI-enabled Proposal Development for an RFP/RFI, P. Nistala et al., 2024, [4]**

##### **Overview**

This paper presents a real-world industrial deployment of an automated proposal generation system for RFP/RFI responses in a large IT organization. The system combines Model-Driven Engineering (MDE) with AI/NLP techniques, successfully deployed to 600+ users across seven geographies, demonstrating significant productivity improvements in proposal development processes.

##### **Methodology**

###### **1. Dataset and Data Source**

The system was deployed in a large Business Unit responding to hundreds of RFPs/RFIs annually. The knowledge base comprised solution documents from 30+ offerings maintained in MS Word and Excel formats, scattered across local and shared repositories. Each RFx typically contains 30-50 questions requiring domain-specific responses. The knowledge model was populated with content from multiple document types including capability documents, methodology descriptions, case studies, past proposals, and technical specifications. After DocToModels processing, the system contained knowledge elements for 30+ domains/offerings, with an average of 30-60 knowledge elements and 10-30 parameters per offering. Over 500+ proposals have been generated using the system across seven geographies, with two major offerings contributing approximately 50% of generated proposals, indicating deep deployment penetration in specific areas.

###### **2. Preprocessing**

The DocToModels engine transformed natural language documents into machine processable models through a pattern-based extraction approach. Documents were analyzed

for structural elements including headings (Heading1, Heading2, Heading3), styles, tables, and formatting. The extraction pattern syntax specified mappings between document structure and the meta-model classes. OpenNLP provided NLP text processing including tokenization, lemmatization, and stop word removal. Docx4j library extracted styled content from Microsoft Office documents. A configurable domain dictionary handled taxonomical variations of domain-specific terms. For RFX processing, question text underwent NLP preprocessing including POS (Part-of-Speech) tagging, stemming, and lemmatization. Questions were classified into Standard/Offering/Project categories, and key metadata including concern (subject area) and knowledge type (information type) were derived using fuzzy matching rules and domain dictionary lookups.

### 3. Model Architecture

The system architecture centered on a domain-agnostic Proposal System Meta-Model with two main components: Proposal Knowledge Model abstracting solution knowledge in terms of Domain, Offering, Concern, KnowledgeElement, KnowledgeType, KnowledgeContext, and KnowledgeParameters, using pattern-based structuring with problem concern, associated context, and knowledge elements; Proposal Solution Model representing RFX, RFXQuestion, RFXContext, K-Map (knowledge mapping), and final Proposal. The system used a reflexive modeling language compatible with OMG MOF (Meta Object Facility) with multiple abstraction levels. The K-Map Recommender implemented the KMatching Algorithm analyzing each RFX question to identify applicable concerns and knowledge types, composing queries, and retrieving relevant knowledge elements filtered by RFX context (geography, customer, market). ModelToText engine composed K-Map recommendations into formatted proposal documents with template selection, question-answer writing following compositional rules, and parameter replacement for context-specific values. A Learning Engine harvested knowledge from submitted proposals using ML-based question classification trained on corpus of questions-concerns pairs from existing model data, performing automated similarity checks and proposing knowledge model updates with human-in-the-loop review.

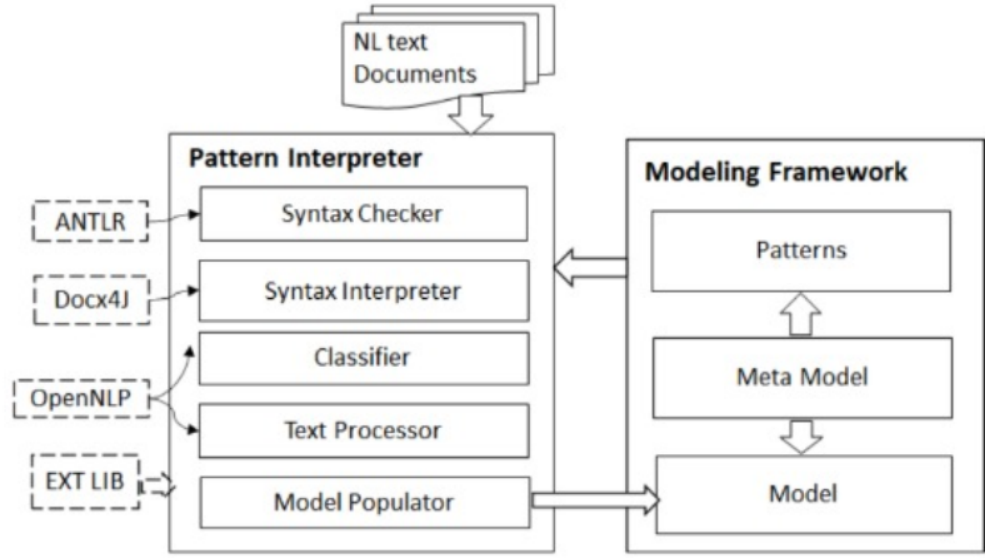


Figure 2.3: Natural Language Document Interpretation Framework (depicts the pattern interpreter modules and their interaction with the modeling framework layers to convert NL text into formal models)[4].

#### 4. Evaluation Metrics

The industrial deployment was evaluated using multiple practical metrics: Time-to-generate proposals comparing manual process (24-48 hours) versus automated system (2-3 hours, or < 0.5 hours for 50-70% completion), Productivity improvement measuring time taken by 10 users before and after system adoption, showing distinct improvements, Coverage metrics including percentage of automatically answered questions (50-70% average), Knowledge density metrics with Mean Knowledge Element Density = 60.5 (sum of KE counts / number of offerings) and Mean Parameter Density = 26.2 (sum of parameter counts / number of offerings), User adoption measuring 600+ active users across seven geographies, and Qualitative feedback from stakeholders on effort automation (70-75%), reduction in human touchpoints, and elimination of manual search/formatting tasks.

#### Results

The system demonstrated strong real-world impact, reaching over 600 users across seven regions, with 18 of 30+ offerings actively using automated proposal generation. It extracted structured content from solution documents, averaging 60.5 knowledge elements and 26.2 parameters per offering. More than 500 proposals were generated, especially

in high-demand areas. Proposal creation time dropped from 24–48 hours to 2–3 hours, with 50–70% of content auto-generated in under 30 minutes. A time study across 10 users confirmed consistent efficiency gains. The K-Map engine accurately matched RFX questions to relevant knowledge using concern matching, classification, and context filtering. Users reported 70–75% effort reduction, improved bid authoring, fewer manual tasks, and enhanced self-service. The system addressed key challenges like consistency, SME dependency, contextual relevance, and formatting. Overall, the deployment validated that AI-driven, model-based approaches can industrialize and automate manual, document-centric workflows.

## **Conclusion**

The paper presents a validated industrial deployment demonstrating practical application of MDE and AI for automating document-intensive business processes. Key contributions include a generic domain-agnostic meta-model, DocToModels approach for transforming NL documents into models, K-Map recommender for intelligent solution recommendations, ModelToText transformation for automated document generation, and Learning Engine for continuous improvement. The deployment demonstrated 70-75% effort reduction and 8-16x productivity improvement, validated through 600+ user deployment. This research is directly applicable to automated proposal generation systems implementation.

## **2.3 Comparison and Gaps Identified**

### **2.3.1 Overview of Surveyed Approaches**

The four papers surveyed represent different but complementary approaches to document understanding, generation, and automation. Table 2.1 provides a comprehensive comparison across multiple dimensions including application domain, core techniques, datasets, evaluation metrics, and key outcomes.

Table 2.1: Comparative Analysis of Surveyed Research Papers

|                         |                                                                                    |                                                                                                                          |                                                                                                        |                                                                                                             |
|-------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Survey Papers           | Financial Report Chunking for Effective Retrieval Augmented Generation (2024), [1] | Unlocking the Potential of Competency Exam Data with Machine Learning: Improving Higher Education Evaluation (2023), [2] | Questioning the Validity of Summarization Datasets and Improving Their Factual Consistency (2022), [3] | An Industrial Experience Report on Model-based, AI-enabled Proposal Development for an RFP/RFI, (2023), [4] |
| Core Technology         | Element-based chunking, Vector databases, GPT-4                                    | MLP, KNN, SVM, Logistic Regression, Decision Trees                                                                       | BART, BERT, RoBERTa, DAE                                                                               | Meta-modeling, Pattern matching, OpenNLP                                                                    |
| Model Architecture      | Weaviate vector DB, Sentence transformers, GPT-4 generator                         | 5 classification algorithms, neural networks, cross-validation                                                           | Pre-trained BART, BERTScore, BARTScore, DAE                                                            | Proposal meta-model, K-Map recommender, ModelToText engine                                                  |
| Primary Metrics         | Q&A accuracy, Page accuracy, ROUGE, BLEU                                           | Accuracy, F1-score, AUC, Recall, Precision                                                                               | BERTScore, BARTScore, DAE, BLANC, ROUGE-2                                                              | Time-to-generate, Coverage %, Productivity improvement, User adoption                                       |
| Best Performance        | 53.19% Q&A accuracy, 84.40% page accuracy, 50% cost reduction                      | MLP: 65.18% accuracy (all-dataset), KNN: 75.37% (micro exam)                                                             | CNN/DM: DAE +0.0072, XLSUM: DAE +0.0425, Maintained ROUGE scores                                       | 70-75% effort reduction, 8-16x faster (24-48hrs to 2-3hrs)                                                  |
| Strengths               | Automatic adaptation, improved retrieval & generation, cost-efficient              | Generalizable framework, multiple algorithms compared, public dataset                                                    | Addresses data quality, improves multiple metrics, systematic approach                                 | Industrial validation, scalable, continuous learning capability                                             |
| Limitations             | Single domain (finance), limited to English, requires document structure           | Specific to education context, requires historical data                                                                  | Relies on automated models, English news only                                                          | Initial setup effort, maintenance overhead, domain dictionary management                                    |
| Relevance to RFP System | RAG implementation, document chunking strategies, retrieval optimization           | Classification for competency matching, ML model selection                                                               | Ensuring factual consistency in generated content                                                      | Direct RFP automation, proven architecture, learning mechanisms                                             |

### 2.3.2 Identified Research Gaps

The comparative analysis reveals key gaps in existing proposal generation systems, including limited support for contextual compliance, lack of real-time personalization, absence of integrated learning from user feedback, and missing end-to-end frameworks that combine retrieval, classification, validation, and deployment. Additionally, current approaches lack business-relevant evaluation metrics beyond standard NLP benchmarks. The proposed system addresses these challenges by integrating element-based RAG for document understanding, machine learning for requirement matching, factual consistency checks, and a model-driven architecture that supports continuous improvement and industrial deployment.

## 2.4 Summary of the Chapter

This chapter reviewed key advancements in automated proposal generation, combining insights from four core research papers and broader industry studies. The first paper demonstrated element-based RAG chunking with 53.19% Q&A accuracy and 50% cost reduction. The second established ML-based competency evaluation, with MLP models achieving 65.18% accuracy via data segmentation. The third introduced SummFC datasets that enhanced factual consistency through systematic filtering. The fourth reported industrial deployment across 600+ users, showing 70–75% effort reduction through model-driven AI/NLP engineering. Complementary studies reinforce these findings that include GPT-based few-shot learning for business text generation [7], NLP-driven proposal management [8], contextual AI for RFP automation [9], document AI for structured data extraction [10], LLMs for context-rich proposals [11].



# Chapter 3

## System Design

### 3.1 Overall System Architecture

The Automated Proposal Creation System is built on a modular, layered architecture to ensure scalability, maintainability, and ease of integration. It comprises three primary layers:

#### 1. Input Layer:

- Accepts RFP documents in multiple formats (PDF, DOCX, TXT).
- Performs initial preprocessing such as format normalization, text extraction, and metadata parsing.
- Ensures document integrity and prepares content for downstream NLP tasks.

#### 2. AI Processing Core:

- RFP Summarization: Uses transformer-based models to extract key sections such as scope, deliverables, evaluation criteria, and deadlines.
- Keyword Extraction: Applies TF-IDF and Named Entity Recognition to identify domain-specific terms and client priorities.
- Scoring & Competency Matching: Compares extracted keywords with organizational capabilities stored in a knowledge base, assigning relevance scores.
- Proposal Generation: Leverages fine-tuned Generative AI models to draft tailored proposal sections including executive summaries, technical approaches, and case studies.

- Feedback Loop: Captures user edits and feedback to refine model outputs over time using reinforcement learning and prompt tuning.

### 3. Output Layer:

- Delivers structured, validated proposals in editable formats.
- Includes compliance checks, grammar correction, and formatting enhancements.
- Supports export to PDF, DOCX, or direct submission portals.

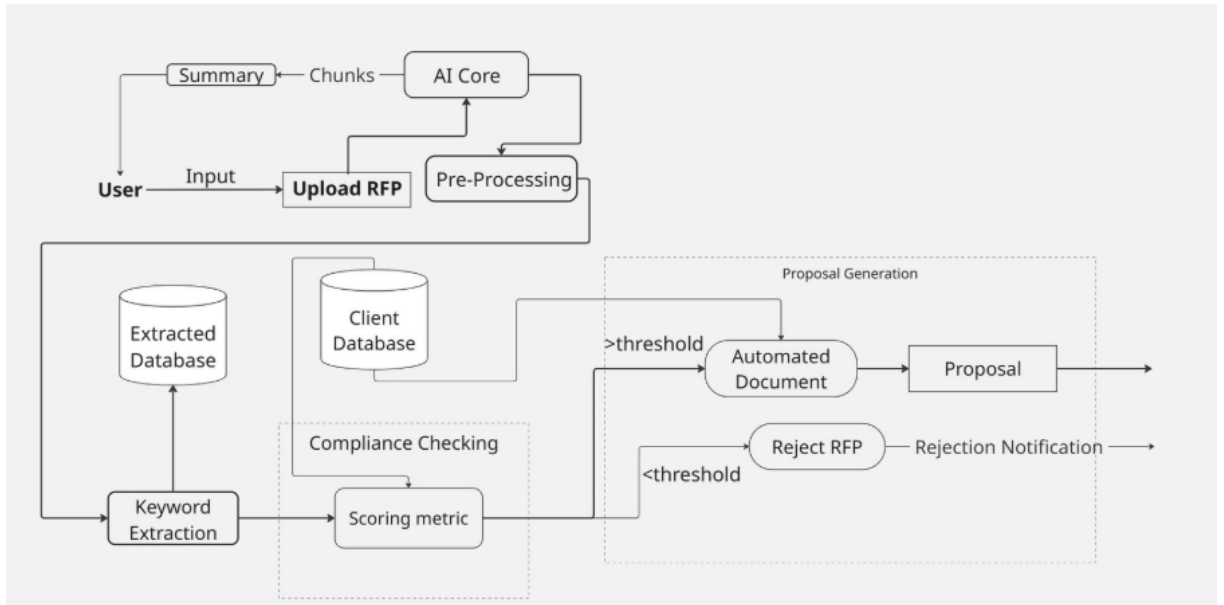


Figure 3.1: Architecture of AI-based RFP Proposal Generation System, showing user input, preprocessing, compliance checking, scoring, and automated proposal generation/rejection flow.

### 3.2 Design Diagrams

The system design is represented through multiple diagrams, including the Architecture Diagram, Use Case Diagram, and Sequence Diagram. The architecture diagram provides an overview of data flow between components. The use case diagram defines interactions between users and system modules, such as uploading RFPs, viewing summaries, and generating proposals. The sequence diagram illustrates the step-by-step process of proposal creation, from document input to validation and final output.

### 3.2.1 Use Case Diagram

This diagram outlines the functional interactions between different actors and system components involved in analyzing an RFP and generating a proposal. It helps clarify who performs what actions and how various processes are interconnected.

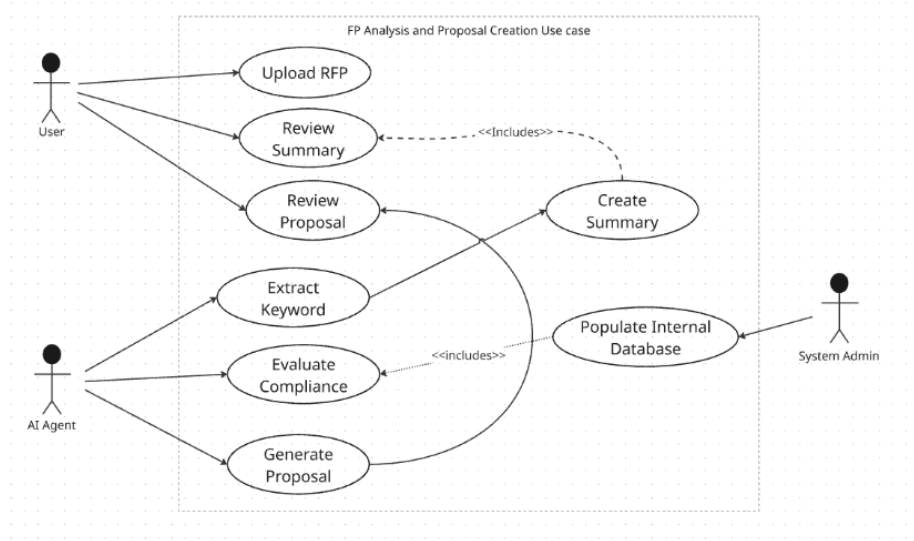


Figure 3.2: Use Case Diagram.

#### 1. Actors Involved

- **User:** The primary actor who interacts with the system to submit RFPs and review outputs.
- **AI Agent:** The intelligent backend component responsible for processing, analyzing, and generating proposals.
- **System Admin:** Oversees the system's internal data and ensures operational integrity.

#### 2. Use Cases Associated with the User

- **Upload RFP:** The user initiates the process by uploading a Request for Proposal document. This triggers the backend pipeline for analysis and generation.

- **Review Summary:** After processing, the user can view a summarized version of the RFP. This summary helps the user quickly understand the scope and requirements.
- **Review Proposal:** The user receives the generated proposal and can review its contents. They may provide feedback or request edits before final submission.

### 3. Use Cases Associated with the AI Agent

- **Extract Keyword:** The AI Agent uses NLP techniques to identify key terms, phrases, and entities from the RFP. These keywords are essential for scoring and matching.
- **Evaluate Compliance:** The AI Agent checks whether the organisation's capabilities align with the RFP requirements. This includes scoring based on keyword relevance and internal data.
- **Generate Proposal:** If compliance is satisfactory, the AI Agent drafts a tailored proposal using Generative AI. The proposal includes sections like executive summary, technical approach, and deliverables.

### 4. Use Cases Associated with the System Admin

- The System Admin maintains and updates the internal competency database.
- This database is used by the scoring engine to evaluate proposal alignment.

### 5. Relationships Between Use Cases

- **Review Summary** includes **Create Summary:** The summary viewed by the user is generated by the system as part of the review process.
- **Evaluate Compliance** includes **Populate Internal Database:** Compliance evaluation relies on data maintained by the System Admin. This ensures that scoring is based on up-to-date organisational capabilities.

### 3.2.2 Sequence Diagram

This diagram outlines the chronological flow of interactions between various system components during the processing of a Request for Proposal (RFP). It captures how data moves from user input to final proposal generation or rejection.

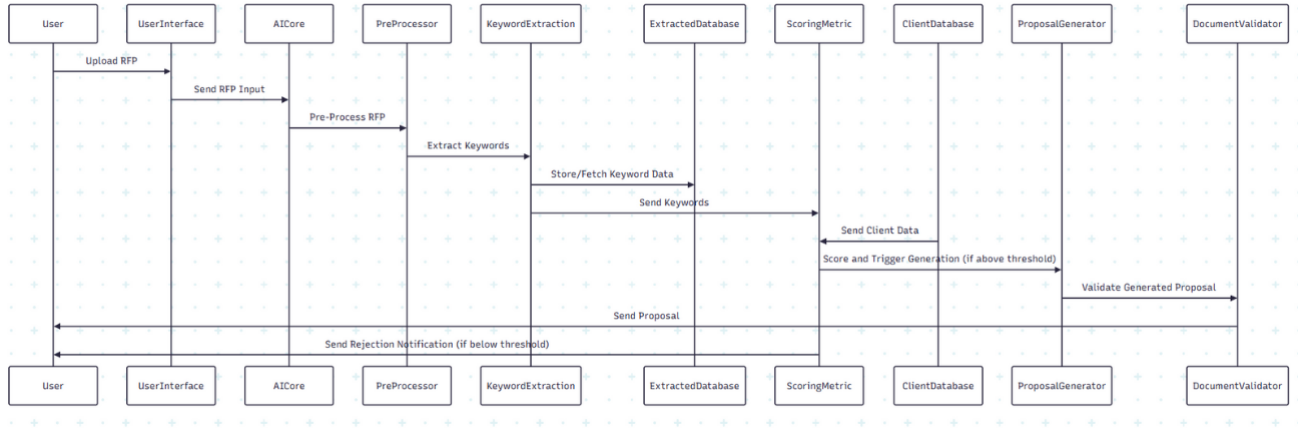


Figure 3.3: Sequence Diagram.

#### 1. User uploads RFP

- The process begins when the user submits an RFP document through the system's user interface.
- This document can be in formats like PDF, DOCX, or plain text.

#### 2. User Interface receives the RFP

- The user interface component accepts the uploaded file and initiates the backend processing.
- It acts as the bridge between the user and the AI-driven backend modules.

#### 3. User Interface sends the RFP to AI core

- The RFP is forwarded to the AI core, which orchestrates the AI-based processing pipeline.
- AI core is responsible for coordinating tasks like preprocessing, keyword extraction, scoring, and generation.

#### **4. AI core triggers PreProcessor**

- The PreProcessor module is activated to clean and normalise the RFP content.
- This includes chunking the document into logical sections and summarising key parts for easier analysis.

#### **5. PreProcessor sends data to Keyword Extractor**

- Once preprocessing is complete, the refined content is passed to the Keyword Extraction module.
- This module uses NLP techniques to identify important terms, phrases, and entities relevant to the RFP.

#### **6. Keyword Extractor interacts with Extracted Database**

- Extracted keywords are stored in a dedicated database for future reference and scoring.
- The module may also retrieve previously stored keywords to enhance context and consistency.

#### **7. Keyword Extractor sends keywords to Scoring Engine**

- The identified keywords are forwarded to the Scoring Engine for evaluation.
- This engine compares the keywords against internal competency data to assess relevance and compliance.

#### **8. Scoring Engine fetches client data from Client Database**

- To perform accurate matching, the Scoring Engine queries the Client Database.
- This database contains organisational capabilities, past performance metrics, and domain expertise.

## **9. Scoring Engine evaluates score**

- A compliance score is calculated based on keyword relevance and alignment with client capabilities.
- If the score meets or exceeds a predefined threshold, the system proceeds to generate a proposal.

## **10. If score is sufficient, Proposal Generator is triggered**

- The Proposal Generator module uses Generative AI to draft a customised proposal.
- It includes sections like executive summary, technical approach, and case studies tailored to the RFP.

## **11. Proposal Generator sends draft to Document Validator**

- The generated proposal is passed to the Document Validator for final checks.
- This module ensures grammatical correctness, formatting consistency, and compliance with RFP guidelines.

## **12. Document Validator completes validation**

- If the proposal passes validation, it is sent back to the User Interface for user review.
- The user can then download, edit, or submit the proposal.

## **13. If score is below threshold, rejection is triggered**

- If the compliance score is too low, the system does not proceed with generation.
- Instead, a rejection notification is sent to the User Interface, informing the user of the outcome.

### 3.2.3 Entity-Relationship Diagram

The ER diagram represents the core database schema for the Automated Proposal Creation System, focusing on the relationship between uploaded documents and the keywords extracted from them. It models a many-to-many relationship between documents and keywords, mediated by a junction table that also stores relevance scores.

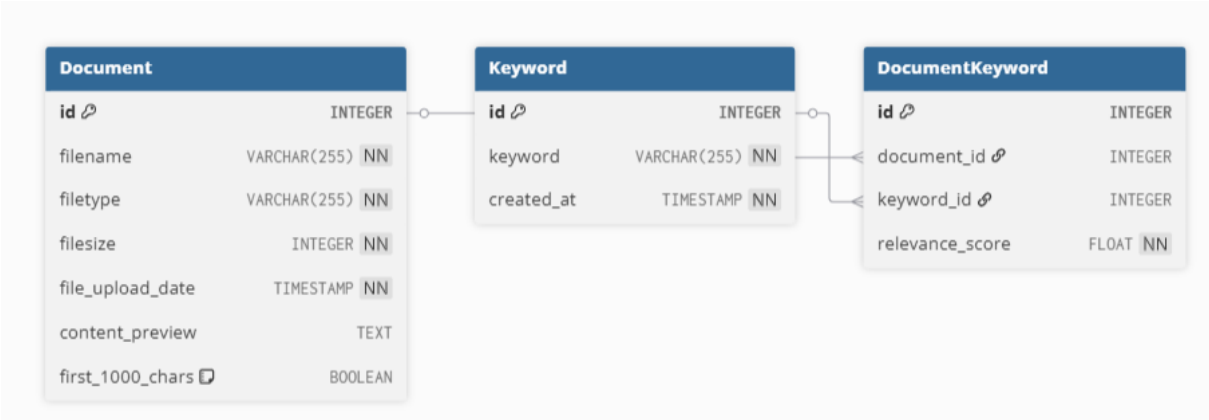


Figure 3.4: Entity-Relationship Diagram.

### 3.3 Component Design

Each module in the system is designed for specific functionality:

- **Input Module:** Accepts RFP documents in multiple formats and forwards them for AI-based analysis.
- **Keyword Extraction Module:** Identifies and stores key phrases for scoring and context mapping.
- **Scoring Metric Module:** Evaluates proposal alignment with RFP criteria and computes compliance scores.
- **Proposal Generation Module:** Uses GenAI to compose coherent, professional proposal drafts that follow organisation-specific templates.
- **Validation and Quality Check:** Ensures proposal consistency, correctness, and compliance with client requirements.



### 3.4 Algorithms Used and Design

The keyword extraction system employs several sophisticated algorithms and design patterns:

#### 3.4.1 KeyBERT Algorithm

KeyBERT is the core algorithm used for keyword extraction. The algorithm works through the following steps:

1. **Document Embedding Generation:** The entire document is converted into a dense vector representation using BERT's sentence transformer model. This creates a semantic fingerprint of the document in a 384-dimensional vector space.
2. **Candidate Keyword Extraction:** Using CountVectorizer with n-gram range of (1,2), the system extracts all possible unigrams and bigrams from the document as candidate keywords.
3. **Candidate Embeddings:** Each candidate keyword is independently converted into its own BERT embedding vector.
4. **Cosine Similarity Calculation:** The algorithm calculates cosine similarity between the document embedding and each candidate keyword embedding using the formula:

$$\text{similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

where  $A$  is the document embedding and  $B$  is the candidate keyword embedding.

5. **MaxSum Diversity Selection:** To ensure diverse keywords, the MaxSum algorithm is applied with diversity parameter of 0.7, which maximises the sum of similarities while minimising redundancy between selected keywords.
6. **Top-N Selection:** The top 15 keywords with highest relevance scores are selected and returned with their corresponding similarity scores.

### 3.4.2 Document Parsing Algorithms

The system implements format-specific parsing algorithms:

- 1. PDF Parsing** Uses PyMuPDF (fitz) library to extract text from PDF documents page by page, handling various PDF encodings and structures.
- 2. DOCX Parsing** Employs python-docx library to extract text from Microsoft Word documents by iterating through paragraphs and preserving document structure.
- 3. TXT Parsing** Simple UTF-8 decoding for plain text files with error handling for encoding issues.

### 3.4.3 Retrieval Augmented Generation (RAG)

Retrieval Augmented Generation is a technique that enhances the output of a large language model by supplying it with relevant external context retrieved from a knowledge source at inference time, rather than relying solely on the model's parametric knowledge. The RAG pipeline in this system operates through the following steps:

- 1. Document Chunking:** At upload time, the full document text is split into 400-word overlapping chunks with a 100-word overlap between consecutive chunks, ensuring that sentences at chunk boundaries are not lost.
- 2. Chunk Embedding Generation:** Each chunk is encoded into a 384-dimensional embedding vector using the all-MiniLM-L6-v2 sentence transformer model and stored in PostgreSQL as a JSON vector field in the DocumentChunk table.
- 3. Query Encoding:** At proposal generation time, a composite query describing the desired proposal content — covering scope, requirements, technical specifications, timeline, and compliance — is encoded into a 384-dimensional vector using the same sentence transformer model.
- 4. Cosine Similarity Retrieval:** Cosine similarity is computed between the query vector and all stored chunk vectors using matrix multiplication over a NumPy array, and the top four most semantically relevant chunks are selected.

5. **Context Assembly:** The retrieved chunks, truncated to 300 characters each to manage prompt length, are concatenated with document metadata including file-name, extracted keywords, budget, and timeline to form the context window passed to the language model.
6. **Grounded Generation:** The assembled context is supplied to LLaMA 3.2 via Ollama as part of a focused prompt for each proposal section, ensuring that the generated content is factually grounded in the actual RFP text rather than relying on generic or hallucinated information.

#### 3.4.4 LLaMA 3.2 via Ollama

LLaMA (Large Language Model Meta AI) is a family of open-source large language models released by Meta AI. The system uses LLaMA 3.2, a decoder-only transformer model with 3 billion parameters, for proposal text generation. The key characteristics of this model and its integration are described below:

1. **Decoder-Only Architecture:** Unlike encoder-only models such as BERT which are designed for text understanding and classification, LLaMA 3.2 is a decoder-only transformer that generates text autoregressively — predicting one token at a time based on all preceding tokens — making it well suited for open-ended text generation tasks such as writing structured proposal sections.
2. **Rotary Position Embeddings (RoPE):** LLaMA 3.2 uses RoPE instead of absolute positional embeddings, enabling better handling of longer contexts by encoding positional information directly into the attention computation.
3. **RMSNorm:** The model uses Root Mean Square Layer Normalisation instead of standard LayerNorm, which improves training stability and reduces computational overhead during inference.
4. **Grouped Query Attention (GQA):** GQA reduces memory usage during inference by sharing key and value heads across multiple query heads, making the model more efficient to run on consumer hardware without a dedicated GPU.

5. **SwiGLU Activation:** The feedforward layers use the SwiGLU activation function, which has been shown to produce higher quality outputs compared to ReLU or GELU in large generative models.
6. **Local Deployment via Ollama:** The model is served locally through Ollama, a tool that packages large language models as locally hosted REST API servers. This eliminates any dependency on external cloud APIs, ensuring complete data privacy and zero inference cost.
7. **Generation Parameters:** Temperature is set to 0.7 to balance coherence with natural variation in phrasing, and a maximum of 600 tokens per section is enforced to manage inference time on consumer hardware, corresponding to approximately 450 words of generated output per proposal section.

#### 3.4.5 System Architecture Design

The system follows a three-tier architecture:

- **Presentation Layer:** Next.js-based frontend with React components for user interaction, including the document upload interface, evaluation results panel, and proposal generation page.
- **Application Layer:** Django REST Framework API handling business logic and document processing, including keyword extraction using the YAKE and all-MiniLM-L6-v2 pipeline, RFP evaluation scoring, and orchestration of the RAG-based proposal generation workflow.
- **AI Inference Layer:** Ollama serving LLaMA 3.2 locally as a REST API, invoked by the application layer during summarization scope extraction and proposal section generation. The all-MiniLM-L6-v2 sentence transformer model runs within the application layer itself for embedding generation, cosine similarity computation, and semantic reranking, requiring no external API calls.
- **Data Layer:** PostgreSQL relational database for persistent storage of documents, extracted keywords, chunk embeddings, evaluation scores, and generated proposals.

### 3.4.6 Design Patterns Implemented

- **Singleton Pattern:** KeywordExtractor class implements singleton pattern to avoid reloading the AI model multiple times, improving performance and memory efficiency.
- **Factory Pattern:** DocumentParser class uses factory pattern to handle different document formats through a unified interface.
- **RESTful API Design:** API endpoints follow REST principles with proper HTTP methods (GET, POST) and resource-based URLs.

## 3.5 Software and Hardware Requirements

This section outlines the essential software dependencies and hardware configurations required to develop, test, and deploy the Automated Proposal Creation System.

### 3.5.1 Software Requirements

#### 1. Backend Technologies

- The system is compatible with major operating systems including Windows 10/11, macOS 10.15+, and Ubuntu 20.04+.
- Python 3.8 or higher serves as the core programming language for backend logic and AI integration.
- Django 5.0.1 provides the web framework for building scalable server-side applications.
- Django REST Framework 3.14.0 enables the creation of secure and modular RESTful APIs.
- PostgreSQL (version 15 or 16) is used as the relational database for storing documents, keywords, and metadata.
- The psycopg2-binary adapter (version 2.9.9) facilitates seamless communication between Django and PostgreSQL.

## **2. AI/ML Libraries**

- KeyBERT 0.8.3 is used for extracting semantically relevant keywords from document text.
- Sentence-transformers 2.2.2 provides pre-trained BERT models for embedding and similarity calculations.
- PyTorch 2.0+ powers the transformer-based models used in keyword extraction and scoring.

## **3. Document Processing Libraries**

- PyMuPDF (fitz) version 1.23.8 handles PDF parsing and text extraction with layout preservation.
- Python-docx 1.1.0 enables structured reading of DOCX files including headings and paragraphs.
- Pillow 10.1.0 supports image manipulation and rendering for document previews.

## **4. Frontend Technologies**

- Node.js 18.0+ serves as the runtime environment for frontend development.
- Next.js 14.0+ is used for building server-rendered React applications with optimised performance.
- React 18.0+ provides the component-based architecture for building dynamic user interfaces.

## **5. Development Tools**

- Visual Studio Code, PyCharm, or similar IDEs are recommended for efficient code editing and debugging.
- pgAdmin 4 is used for managing and querying the PostgreSQL database.
- Postman or Thunder Client are employed for testing API endpoints and validating responses.

- Git is used for version control, enabling collaborative development and change tracking.

### **3.5.2 Hardware Requirements**

#### **Minimum Requirements**

- Processor: Intel Core i3 or equivalent processor.
- RAM: 4 GB (sufficient for basic functionality and single-user operation).
- Storage: 10 GB of free disk space for operating system, development tools, Python packages, and pre-trained AI models.

#### **Recommended Requirements**

- Processor: Intel Core i5/i7 or AMD Ryzen 5/7 for faster document processing and multitasking.
- RAM: 8 GB or higher to ensure efficient memory management and faster keyword extraction.
- Storage: 20 GB of free space, preferably on a Solid State Drive (SSD) for improved application load times and database query performance.
- GPU: Optional NVIDIA GPU with CUDA support for accelerating machine learning operations (beneficial for large-scale processing).

### **3.6 Dataset Identified**

Unlike traditional machine learning approaches, this project utilises pre-trained BERT models, specifically the all-MiniLM-L6-v2 model from the sentence-transformers library. This model was originally trained on over 1 billion sentence pairs and multiple datasets including MS MARCO (Microsoft Machine Reading Comprehension), Natural Questions dataset, SQuAD (Stanford Question Answering Dataset), and various web crawl data.

The document corpus comprises three primary types of files: PDF, DOCX, and TXT. PDF documents are the most prevalent, with over 50 files, each averaging between 3000 and 8000 words. DOCX documents follow with more than 30 files, typically ranging from

1500 to 5000 words in length. The corpus also includes over 20 TXT files, which are generally shorter, averaging between 500 and 3000 words.

### 3.7 Database

#### 3.7.1 Database Management System

The system uses PostgreSQL 16, an advanced open-source relational database management system, chosen for its ACID compliance ensuring data integrity, support for complex queries and indexing, scalability for growing data requirements, JSON support for flexible data structures, and strong community support.

#### 3.7.2 Database Schema

The database consists of three main tables listed below.

##### 1. Document Table (`generation_document`)

| Column          | Type         | Constraints   | Description       |
|-----------------|--------------|---------------|-------------------|
| id              | INTEGER      | PRIMARY KEY   | Unique identifier |
| filename        | VARCHAR(255) | NOT NULL      | Original filename |
| file            | VARCHAR(100) | NOT NULL      | File path         |
| file_type       | VARCHAR(10)  | NOT NULL      | File extension    |
| upload_date     | TIMESTAMP    | NOT NULL      | Upload timestamp  |
| content_preview | TEXT         | NULL          | First 1000 chars  |
| processed       | BOOLEAN      | DEFAULT FALSE | Processing status |

Table 3.1: Document Table Schema. Stores uploaded document metadata and content preview.



## 2. Keyword Table (`generation_keyword`)

| Column     | Type         | Constraints      | Description        |
|------------|--------------|------------------|--------------------|
| id         | INTEGER      | PRIMARY KEY      | Unique identifier  |
| keyword    | VARCHAR(255) | UNIQUE, NOT NULL | Keyword text       |
| created_at | TIMESTAMP    | NOT NULL         | Creation timestamp |

Table 3.2: Keyword Table Schema. Stores unique keywords extracted across all documents.

## 3. DocumentKeyword Table (`generation_documentkeyword`)

| Column          | Type    | Constraints | Description            |
|-----------------|---------|-------------|------------------------|
| id              | INTEGER | PRIMARY KEY | Unique identifier      |
| document_id     | INTEGER | FOREIGN KEY | Reference to document  |
| keyword_id      | INTEGER | FOREIGN KEY | Reference to keyword   |
| relevance_score | FLOAT   | NOT NULL    | Similarity score (0–1) |

Table 3.3: DocumentKeyword Junction Table Schema. Links documents with keywords and relevance scores.

### 3.7.3 Database Indexing Strategy

- Primary Key Indexes: Automatically created on all `id` columns.
- Unique Index: On `keyword.keyword` for fast duplicate checking.
- Foreign Key Indexes: On `document_id` and `keyword_id` for efficient joins.
- Composite Index: On (`document_id`, `relevance_score`) for sorted retrieval.

## 3.8 Budget

### 3.8.1 Cost Analysis

This project utilises entirely open-source and free technologies, resulting in zero software licensing costs. However, for deployment and scalability considerations, the following

budget analysis is provided:

### Development Phase (Current)

| Resource                | Cost             | Notes                                        |
|-------------------------|------------------|----------------------------------------------|
| Software Tools          | Rs. 0            | All open-source                              |
| Development Hardware    | Rs. 0            | Existing computers                           |
| Internet                | Rs. 500/month    | Existing connection                          |
| Documentation Tools     | Rs. 0            | Free L <sup>A</sup> T <sub>E</sub> X editors |
| <b>Total (3 months)</b> | <b>Rs. 1,500</b> |                                              |

Table 3.4: Development Phase Budget

### Deployment Phase (Future)

| Resource                | Monthly Cost             |
|-------------------------|--------------------------|
| Cloud Hosting (Backend) | Rs. 500–Rs. 2000         |
| Database Hosting        | Rs. 300–Rs. 500          |
| Frontend Hosting        | Rs. 0                    |
| <b>Total (Monthly)</b>  | <b>Rs. 800–Rs. 3,500</b> |

Table 3.5: Deployment Phase Budget Estimate

#### 3.8.2 Cost-Benefit Analysis

- Development Cost: Minimal due to open-source stack.
- Operational Cost: Scalable based on usage.
- Maintenance Cost: Low due to mature, well-documented technologies.
- ROI Potential: High automation value for document processing tasks.

## 3.9 Module Division and Work Responsibilities

### 3.9.1 Module Breakdown

#### Module 1: Document Processing Module

##### Components:

- PDF parser (PyMuPDF)
- DOCX parser (python-docx)
- TXT parser
- File validation and error handling

**Responsibilities:** The Document Processing Module serves as the foundation of the system by handling the extraction of text content from various document formats including PDF, DOCX, and TXT files. This module implements robust parsing mechanisms that can handle encoding errors gracefully while preserving the original document structure and formatting. The module also incorporates comprehensive file validation to ensure that only supported file formats are processed, along with sophisticated error handling mechanisms that provide meaningful feedback when issues are encountered during the document parsing phase.

#### Module 2: Keyword Extraction Module

##### Components:

- KeyBERT integration
- BERT model initialisation (Singleton pattern)
- Embedding generation
- Similarity calculation
- Diversity-based selection

**Responsibilities:** The Keyword Extraction Module represents the core intelligence of the system, leveraging advanced AI-powered semantic analysis through KeyBERT to identify and extract the most relevant keywords from processed documents. This module initialises and manages BERT models using the Singleton design pattern to ensure efficient resource utilisation across multiple extraction requests. The module generates contextualised embeddings for document text, calculates semantic similarity scores between potential keywords and the document content, and employs diversity-based selection algorithms to ensure that extracted keywords provide comprehensive coverage of the document's themes while avoiding redundancy.

### Module 3: Backend API Module

#### Components:

- Django REST Framework setup
- API endpoints (upload, retrieve, delete)
- Request validation and serialisation
- Error handling and logging
- CORS configuration
- Ollama REST API client for local LLaMA inference
- RAG chunk retrieval and cosine similarity computation

**Responsibilities:** The Backend API Module establishes a robust RESTful interface that facilitates seamless communication between the frontend user interface and the backend processing systems. Built on Django REST Framework, this module defines and implements all necessary API endpoints for document upload, retrieval, and deletion operations. The module incorporates comprehensive request validation to ensure data integrity, serialisation mechanisms for efficient data transformation, and sophisticated error handling with detailed logging capabilities to aid in debugging and system monitoring.

## Module 4: Database Module

### Components:

- Database models (Document, Keyword, DocumentKeyword)
- PostgreSQL connection configuration
- Migration management
- Query optimisation

**Responsibilities:** The Database Module manages all aspects of data persistence and retrieval within the system through a well-structured relational database design. This module defines and maintains three primary models: Document for storing document metadata and content, Keyword for managing extracted keywords, and DocumentKeyword for maintaining the many-to-many relationships between documents and their associated keywords. The module handles PostgreSQL database connection configuration and credentials management, oversees database schema migrations to ensure smooth evolution of the data structure, and implements query optimisation strategies including appropriate indexing.

## Module 5: Frontend UI Module

### Components:

- Document upload interface
- Keyword visualisation components
- Document history view
- Responsive design implementation
- API integration layer

**Responsibilities:** The Frontend UI Module delivers an intuitive and engaging user experience through a carefully designed web interface that enables users to upload documents and visualise extracted keywords with ease. This module implements a modern document upload interface with drag-and-drop functionality, sophisticated keyword visualisation components that display relevance scores using colour-coded indicators, and a comprehensive document history view that allows users to track their previous uploads and results.

## **Module 6: Admin Module**

### **Components:**

- Django admin panel configuration
- Custom admin views
- Data management interface

**Responsibilities:** The Admin Module provides system administrators with powerful tools for managing and monitoring the application through a customised Django admin interface. This module extends the default Django admin panel with custom views tailored specifically to the needs of the keyword extraction system, enabling administrators to efficiently manage documents, keywords, and user data.

## **Module 7: RFP Evaluation Module**

### **Components:**

- Technical fit scoring engine
- Budget fit scoring engine
- Timeline fit scoring engine
- Overall fit calculation
- Decision threshold logic

**Responsibilities:** The RFP Evaluation Module automatically assesses the compatibility between an uploaded RFP document and the company’s capabilities by computing three independent fit scores. The technical fit score is calculated through a four-layer keyword matching process — exact string matching, substring matching, full-text search, and word overlap detection — followed by a semantic boost using the all-MiniLM-L6-v2 sentence transformer model. The final technical fit score is a weighted blend of 70% keyword matching and 30% semantic similarity. The budget fit score checks whether the RFP’s detected budget falls within the company’s acceptable project range stored in the CompanyCapability profile. The overall fit score is computed as a weighted average — technical fit carries 50% weight, while budget and timeline each carry 25%. Based on the overall score, the system assigns a decision: ACCEPT for scores at or above 40%, REVIEW for scores at or above 20%, and REJECT for scores below 20%.

## Module 8: Proposal Generation Module

### Components:

- RAG retrieval pipeline
- LLaMA 3.2 via Ollama integration (local)
- Gemini API integration (comparison)
- Section parser
- Proposal display interface

**Responsibilities:** The Proposal Generation Module produces a complete, structured bid proposal in response to an accepted RFP using a Retrieval Augmented Generation (RAG) pipeline. When a user triggers proposal generation, the system retrieves the four most semantically relevant chunks from the indexed document by encoding a query using the all-MiniLM-L6-v2 model and computing cosine similarity against all stored chunk embeddings in PostgreSQL. The system generates six sections: Executive Summary, Technical Approach and Methodology, Timeline and Milestones, Budget Breakdown, Proposed Team Structure, and Compliance Checklist. Two backends are supported: LLaMA 3.2 running locally via Ollama, and the Gemini 2.0 Flash API.

### 3.9.2 Work Distribution

| Module                                            | Team Member              |
|---------------------------------------------------|--------------------------|
| Document Processing & Parsing                     | P.A. Abidh Muhammed      |
| Keyword Extraction & Metadata Extraction          | Niranjana N              |
| Backend API & Proposal Generation (Local-LLaMA)   | Niranjana N              |
| Database Design & Company Profile Setup           | Rahul Rathishkumar Menon |
| Frontend UI & Evaluation Display                  | Rahul Rathishkumar Menon |
| Summary Generation & Proposal Generation (Gemini) | Nevin John George        |
| Testing & Performance Analysis                    | Nevin John George        |
| Documentation                                     | P.A. Abidh Muhammed      |

Table 3.6: Work Distribution Among Team Members

### 3.10 Key Deliverables – Expected Outputs

This section outlines the tangible outputs expected from the development of the Automated Proposal Creation System. Deliverables are categorised into primary and secondary outputs.

#### 3.10.1 Software Deliverables

##### Primary Deliverables

##### 1. Functional Web Application

- A fully operational keyword extraction system that processes uploaded RFP documents using NLP techniques.
- Support for multiple document formats including PDF, DOCX, and TXT.
- Real-time processing and keyword display, allowing users to receive immediate feedback upon document upload.
- Document history management, enabling users to track previously uploaded files and revisit keyword results.



## 2. RESTful API

- `POST /api/documents/upload/`: Endpoint for uploading new RFP documents.
- `GET /api/documents/`: Retrieves a list of all uploaded documents with metadata.
- `GET /api/documents/{id}/`: Fetches detailed information about a specific document.
- `GET /api/documents/{id}/keywords/`: Returns extracted keywords with relevance scores.
- `POST /api/documents/{id}/generate_proposal/`: Triggers RAG-based proposal generation.
- `GET /api/documents/{id}/evaluation/`: Returns fit scores, overall score, and decision.

## 3. Database System

- A properly normalised schema that minimises redundancy and ensures efficient data organisation.
- Indexed tables to accelerate query performance for keyword lookups and document retrieval.
- Data integrity constraints such as primary keys, foreign keys, and validation rules.

## 4. User Interface

- A responsive layout that adapts seamlessly to desktops, tablets, and mobile screens.
- Drag-and-drop file upload functionality for ease of use.
- Visual keyword display with relevance scores.
- A document history view providing access to previously processed files.

## **5. RFP Evaluation System**

- Automated technical fit scoring using a four-layer keyword matching algorithm combined with semantic similarity.
- Budget and timeline fit scoring based on the company's acceptable project range.
- Overall fit score with a clear ACCEPT, REVIEW, or REJECT decision.
- Reasoning text explaining each score.

## **6. AI-Powered Proposal Generation**

- RAG retrieval using MiniLM embeddings stored in PostgreSQL.
- LLaMA 3.2 running locally via Ollama generates six proposal sections.
- Gemini 2.0 Flash API integration for comparative proposal generation.
- Generated proposals saved to the database and displayed with collapsible sections, TXT download, and print to PDF.

## **Secondary Deliverables**

### **1. Source Code**

- Well-commented Python and JavaScript code that explains logic and usage.
- A modular architecture that separates concerns and facilitates future enhancements.
- Version control using Git, enabling collaborative development and change tracking.

### **2. Documentation**

- The project report detailing design decisions, implementation strategies, and evaluation metrics.
- API documentation describing available endpoints, request formats, and response structures.
- A user manual guiding non-technical users through system features and workflows.

- An installation guide for setting up the system in local or cloud environments.
- Inline code documentation using docstrings and comments.

### 3. Testing Suite

- Unit tests for core functions such as document parsing, keyword extraction, and scoring.
- API endpoint tests to verify correct responses, error handling, and data integrity.
- Sample test documents used to simulate real-world scenarios and benchmark performance.

#### 3.10.2 Expected Performance Metrics

| Metric                                  | Target Value                      |
|-----------------------------------------|-----------------------------------|
| Document Processing Time                | < 5 seconds (for 10-page PDF)     |
| Keyword Extraction Time (first request) | < 10 seconds                      |
| Keyword Extraction Time (subsequent)    | < 3 seconds                       |
| API Response Time                       | < 1 second (excluding processing) |
| Supported File Size                     | Up to 10 MB                       |
| Concurrent Users                        | 5–10 users                        |
| Keyword Accuracy                        | 80–90% relevance                  |
| System Uptime                           | 99% (in production)               |

Table 3.7: Expected Performance Metrics

#### 3.10.3 Output Formats

The system produces outputs in multiple formats. JSON API responses provide structured keyword data with associated relevance scores. The web interface presents keywords as interactive pill-shaped chips with hover effects showing relevance scores. RFP evaluation results are displayed as animated progress bars for technical, budget, and timeline fit

scores alongside a clear ACCEPT, REVIEW, or REJECT decision badge with explanatory reasoning text. Generated proposals are displayed in a collapsible section-by-section interface and can be downloaded as a plain text file or printed to PDF directly from the browser.

### 3.11 Project Timeline

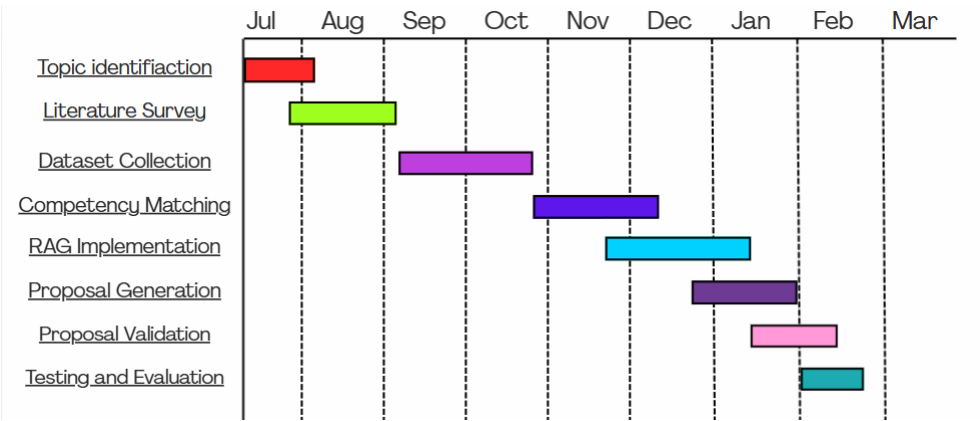


Figure 3.5: Project Gantt Chart.

### 3.12 UI Design

#### 3.12.1 Design Principles

The user interface has been developed following modern web design principles that prioritise both aesthetics and functionality. The minimalist design philosophy ensures a clean and uncluttered interface that focuses users' attention on the core functionality of document upload and keyword extraction. Responsive layout implementation guarantees that the interface adapts seamlessly to various screen sizes and device types. The interface features intuitive navigation with self-explanatory controls, visual feedback mechanisms, and accessibility considerations including appropriate colour contrast ratios and readable font selections.

### **3.12.2 UI Components**

#### **Main Upload Interface**

The main upload interface serves as the primary entry point for user interaction with the system. It features a prominent drag-and-drop file upload area that accepts PDF, DOCX, and TXT files up to 10MB, allowing users to either drag documents directly from their file explorer or click the upload zone to open a standard file selection dialog. File format and size validation occurs immediately upon file selection, providing instant visual feedback by displaying the selected filename within the upload zone along with a confirmation indicator. A clearly labelled Upload and Evaluate button initiates the full processing pipeline upon click, triggering document parsing, keyword extraction, summarization, metadata extraction, RAG indexing, and fit scoring in a single automated sequence. A reset button is provided to clear the current file selection and return the interface to its initial state, enabling users to process a different document without refreshing the page. During processing, a loading spinner is displayed to communicate active pipeline execution status to the user.

#### **Keyword Display Interface**

The keyword display interface presents the top 15 extracted keywords as interactive pill-shaped chips arranged in a flowing horizontal layout. Each chip displays the keyword text alongside its relevance score expressed as a percentage, providing users with a quantitative indication of each term's contextual significance within the document. The chips are ordered in descending relevance score, ensuring the most important terms are immediately visible. Hover effects on each chip provide subtle visual feedback, improving the overall interactivity of the interface. The keyword set displayed reflects the output of the two-stage YAKE and MiniLM reranking pipeline, capturing domain-specific terminology relevant to the RFP's subject matter while filtering out generic noise words.

#### **Document Information Panel**

The document information panel provides a comprehensive view of all processing outputs for the uploaded document, organised into distinct subsections. The metadata block displays the original filename, upload timestamp, detected file type, and total count of

extracted keywords, allowing users to confirm the document was correctly identified and processed. The document summary subsection presents the hybrid regex and LLaMA-generated structured summary, including the issuer name, RFP purpose, scope of work description, and key facts such as estimated project value, EMD, bid validity, and submission deadline. The evaluation subsection presents three animated progress bars corresponding to the technical fit score, budget fit score, and timeline fit score, each expressed as a percentage. A prominently displayed overall fit score is accompanied by a colour-coded decision badge — green for ACCEPT, amber for REVIEW, and red for REJECT — along with a plain-language reasoning block that explains the basis of each score, identifying matched and unmatched keywords and whether the budget and timeline fell within acceptable ranges. When the decision is ACCEPT, an additional green banner is rendered at the top of the results area presenting a Generate Proposal button that navigates the user to the proposal generation page.

### **3.13 Summary of Chapter**

This chapter presents the system design of the Automated Proposal Creation System, including its algorithms, technical architecture, and implementation strategy. The system uses a YAKE-inspired statistical method with all-MiniLM-L6-v2 sentence transformer embeddings for semantic keyword extraction, replacing KeyBERT to reduce dependency overhead and improve speed. It supports PDF, DOCX, and TXT document parsing using PyMuPDF and python-docx. The technology stack includes Django REST Framework, PostgreSQL, sentence-transformers, Ollama with LLaMA 3.2, and Next.js. A normalised database schema ensures efficient storage and retrieval of documents, keywords, embeddings, evaluations, and proposals. The system consists of eight modules: document processing, keyword extraction, backend API, database, frontend UI, admin, RFP evaluation, and proposal generation. Proposal generation uses RAG with MiniLM embeddings and LLaMA 3.2 via Ollama for local, privacy-preserving generation.

## Chapter 4

### Results

#### 4.1 Introduction

This chapter presents the empirical results obtained from the fully implemented Automated Proposal Generation System. Each section corresponds to a functional module in the processing pipeline, covering document upload and parsing, keyword extraction, document summarization, RFP metadata extraction, competency fit evaluation, and AI-driven proposal generation. Results are supported by screenshots of the live system interface, demonstrating the output of each processing stage on a real government RFP document used as the primary test case throughout evaluation.

#### 4.2 Document Upload and Processing

The document upload interface accepts files in PDF, DOCX, and TXT formats with a maximum file size of 10MB, covering the majority of government and enterprise RFP document formats encountered in practice. Upon file selection, the system performs immediate client-side format validation and displays the filename within the upload zone, providing the user with confirmation before submission. Clicking the Upload and Evaluate button initiates the full backend processing pipeline, during which a loading spinner communicates active processing status to the user.

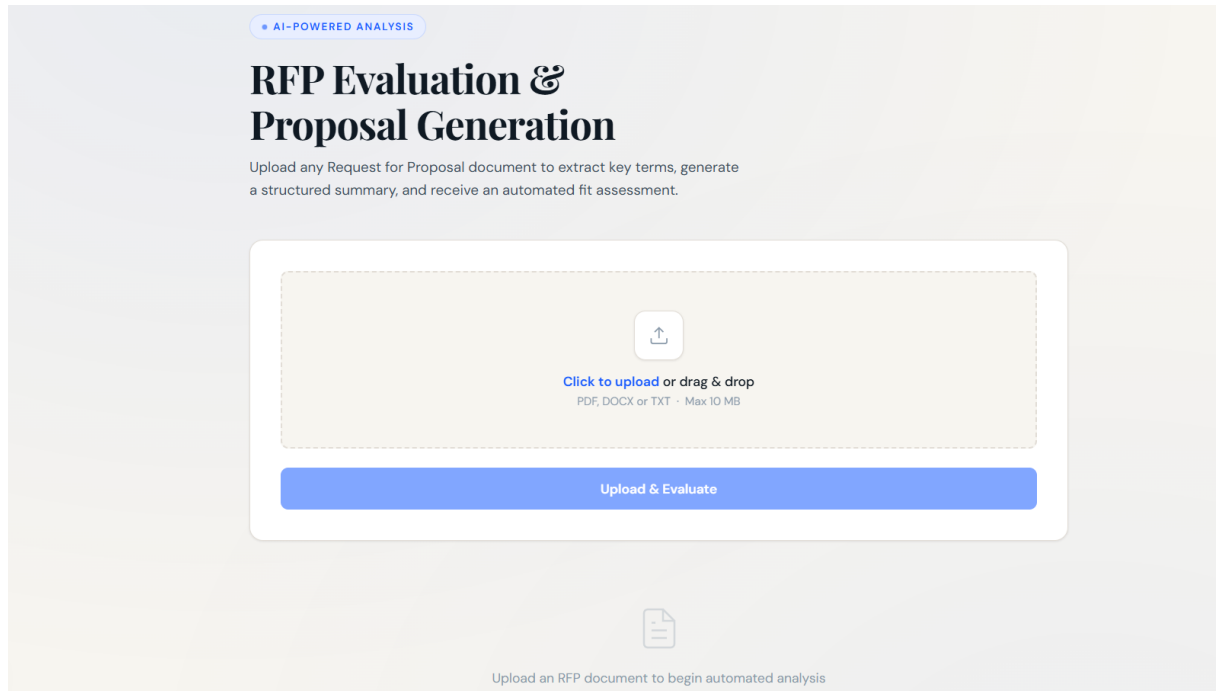


Figure 4.1: Document Upload Interface showing file selection and Upload and Evaluate button.

Internally, the pipeline executes the following sequential operations: raw text extraction using PyMuPDF for PDF files and python-docx for DOCX files, tokenisation and chunk splitting into 400-word overlapping segments for RAG indexing, YAKE-based keyword scoring followed by semantic reranking, hybrid summarization combining regex extraction with an Ollama-generated scope description, regex-based metadata extraction for budget, EMD, timeline, and team size, batch embedding of all document chunks using the all-MiniLM-L6-v2 sentence transformer, storage of embeddings in PostgreSQL as JSON vectors, and computation of technical, budget, and timeline fit scores against the stored company capability profile. The complete pipeline for a typical government RFP document of approximately 30 pages completes within 30 to 60 seconds on a standard consumer laptop without GPU acceleration.

### 4.3 Keyword Extraction Results

The keyword extraction module implements a two-stage pipeline. In the first stage, a YAKE-inspired statistical scoring algorithm evaluates each candidate term — comprising unigrams, bigrams, and trigrams, based on four factors: term frequency within the doc-



ument, positional weight favouring terms appearing earlier in the text, dispersion score reflecting how evenly the term is distributed across the document, and a case bonus applied to acronyms and capitalised domain terms. An extended stopwords list filters generic RFP noise words such as “clause”, “section”, “pursuant”, “accordance”, and “any” that carry no semantic value despite high frequency. In the second stage, the top 50 YAKE candidates are encoded alongside the full document embedding using the all-MiniLM-L6-v2 sentence transformer, and cosine similarity scores between each keyword embedding and the document embedding are computed. The final relevance score for each keyword is a weighted blend of 40% YAKE score and 60% semantic similarity, ensuring that extracted keywords are both statistically significant and contextually central to the document.

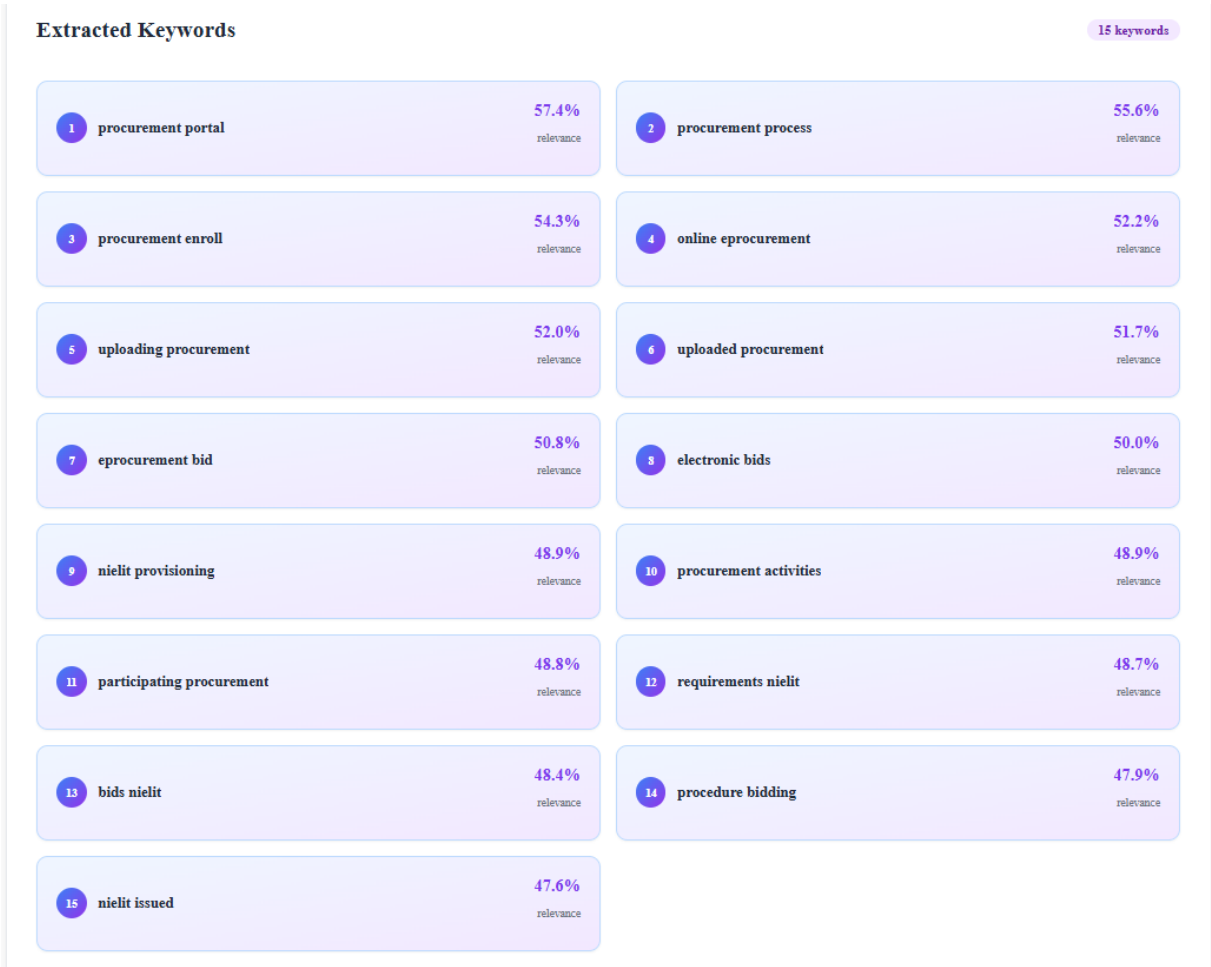


Figure 4.2: Extracted Keywords Display showing the top 15 semantically ranked keywords with relevance scores.

The top 15 keywords are displayed in the interface as interactive pill-shaped chips

ordered by descending relevance score, each annotated with its percentage score. The extracted keyword set accurately reflects the core domain topics of the uploaded RFP, capturing domain-specific terminology related to the procurement process, technical requirements, and institutional context. These keywords directly inform the downstream technical fit scoring process and are passed as contextual metadata to the proposal generation prompts.

#### **4.4 Document Summary Results**

The summarization module produces a structured prose summary through a hybrid extraction and generation approach designed to maximise both accuracy and readability. The module operates in three stages. First, the issuer name is extracted from the first ten lines of the document, where government RFPs consistently place the issuing organisation’s name. Second, a set of regex patterns targeting standard government RFP phrasing extracts the purpose of the RFP along with structured facts including estimated project value, EMD or bid security amount, bid validity period, and submission deadline. Third, to avoid the fragmented scope descriptions produced by purely regex-based approaches — which tend to capture mid-sentence fragments from tables and numbered lists — the scope of work sentence is generated by LLaMA 3.2 running locally via Ollama, using the first 3000 characters of the document as context with a low temperature setting of 0.1 to produce a factual, grounded description of the expected deliverables.



## Document Summary

The National Institute of Electronics and Information Technology (NIELIT), under the Ministry of Electronics and IT (MeitY), Government of India, issued a Request for Proposal (RFP) on June 10, 2017. This RFP seeks a competent and professional bidder for the **Design, Development, and Implementation of an Online Integrated Student Life Cycle Management System**. **Main Objectives:** The overarching objective is to establish a scalable, secure, robust, and responsive web-enabled system for centralized student lifecycle management. This system aims to be flexible enough to adapt to evolving policies and administrative rules, streamline processes from institute accreditation to student placement, and integrate with existing NIELIT systems (e.g., online examination, e-learning, e-certification, placement portal). Key goals include providing transparent online accreditation, standardizing course structures, centralizing registration, offering a common student interface for inquiries and enrollment, and generating comprehensive MIS reports for government departments like MeitY and the National Skill Development Agency (NSDA) through its Labour Market Information System (LMIS). Ultimately, the initiative seeks to minimize duplication, paperwork, and manual data transfer, allowing NIELIT officials to focus on core activities and improve data management and retrieval mechanisms. **Key Methods & Scope:** The project outlines a comprehensive, phased approach: 1. **Analysis:** Involves preliminary study of existing NIELIT systems, detailed requirement gathering, and finalization of software, database configurations, and data migration strategies. 2. **Design:** Development of Software Requirements Specification (SRS), Functional Requirement Specification (FRS), high-level and low-level application designs, prototyping, data backup, and Disaster Recovery Management (DRM) policies, ensuring compliance with security guidelines from CERT-In, NIC, and MeitY. 3. **Development:** Coding and application development, integration with NIELIT's online payment gateway, adherence to web application audit and security features (e.g., OWASP vulnerability-free, strong password policy, session management, audit trails), high availability architecture, and rigorous multi-stage testing (Unit, Integration, System, Functional, Peak Load, UAT). 4. **Operation & Maintenance:** Identifying and executing training requirements for NIELIT staff, creation of user/technical manuals, and providing post-implementation support, including a one-year warranty and a mandatory offer for a two-year Annual Maintenance Contract (AMC). The system must support at least 1000 concurrent users, be initially bilingual (English and Hindi) with future multilingual provision, and be deployable on the NIC Cloud (Meghraj). The bidder is responsible for a CERT-In security audit and migration of existing and historical data. Functional modules will include Accreditation (defining rules, online applications, scrutiny, inspection, renewal, withdrawal), Expert Management (empanelment, duty assignment, claims processing), Course Management (adding/revising courses, defining rules and dependencies), and Student Information and Registration (unique IDs, online enrollment for all course types, batch uploads, re-registration, transfers, exam scheduling). **Important Results & Conclusions:** The successful implementation will yield a centralized, highly secure, and efficient platform for managing NIELIT's extensive educational and training ecosystem. It will ensure transparency, reduce administrative overhead, and enable compliance with national reporting standards. A critical outcome is that NIELIT will be the sole proprietor of the developed system's Intellectual Property Rights (IPR) and receive the complete, error-free source code. The project underscores NIELIT's commitment to modernizing its operations through advanced, integrated IT solutions, ensuring robust support and continuous improvement.

Figure 4.3: Document Summary panel showing the structured summary.

The resulting summary presents the issuer name, the stated purpose of the RFP, a concise scope of work description, and a set of structured bullet points covering the estimated project value, EMD, bid validity, and submission deadline. This structured format enables users to rapidly assess the key parameters of an RFP without reading the full document, significantly reducing the time required for initial evaluation.

## 4.5 RFP Metadata Extraction Results

The metadata extraction module operates independently of the summarization pipeline, applying dedicated regex patterns to the raw document text to identify four structured fields: project budget or estimated cost, EMD or bid security amount, project timeline expressed in weeks, and required team size. Each pattern is designed to handle the varied phrasing conventions used across different government RFP formats, including variations such as “estimated cost”, “contract value”, “approximate value”, and “project value” for the budget field, and “earnest money deposit”, “EMD”, and “bid security” for the security deposit field.

The extraction confidence level is reported as one of four categories: complete when all four fields are successfully extracted, partial when two or three fields are found, minimal when only one field is found, and insufficient when no fields can be extracted. The extracted budget and timeline values serve as direct inputs to the budget fit and timeline fit scoring functions in the evaluation module, making accurate metadata extraction critical to the reliability of the overall fit assessment.

## 4.6 RFP Evaluation Results

The evaluation module computes three independent fit scores by comparing the extracted RFP parameters against the company capability profile stored in the CompanyCapability model in the database. The company profile contains technical keywords spanning relevant technology domains, along with a configured acceptable budget range and project timeline range.

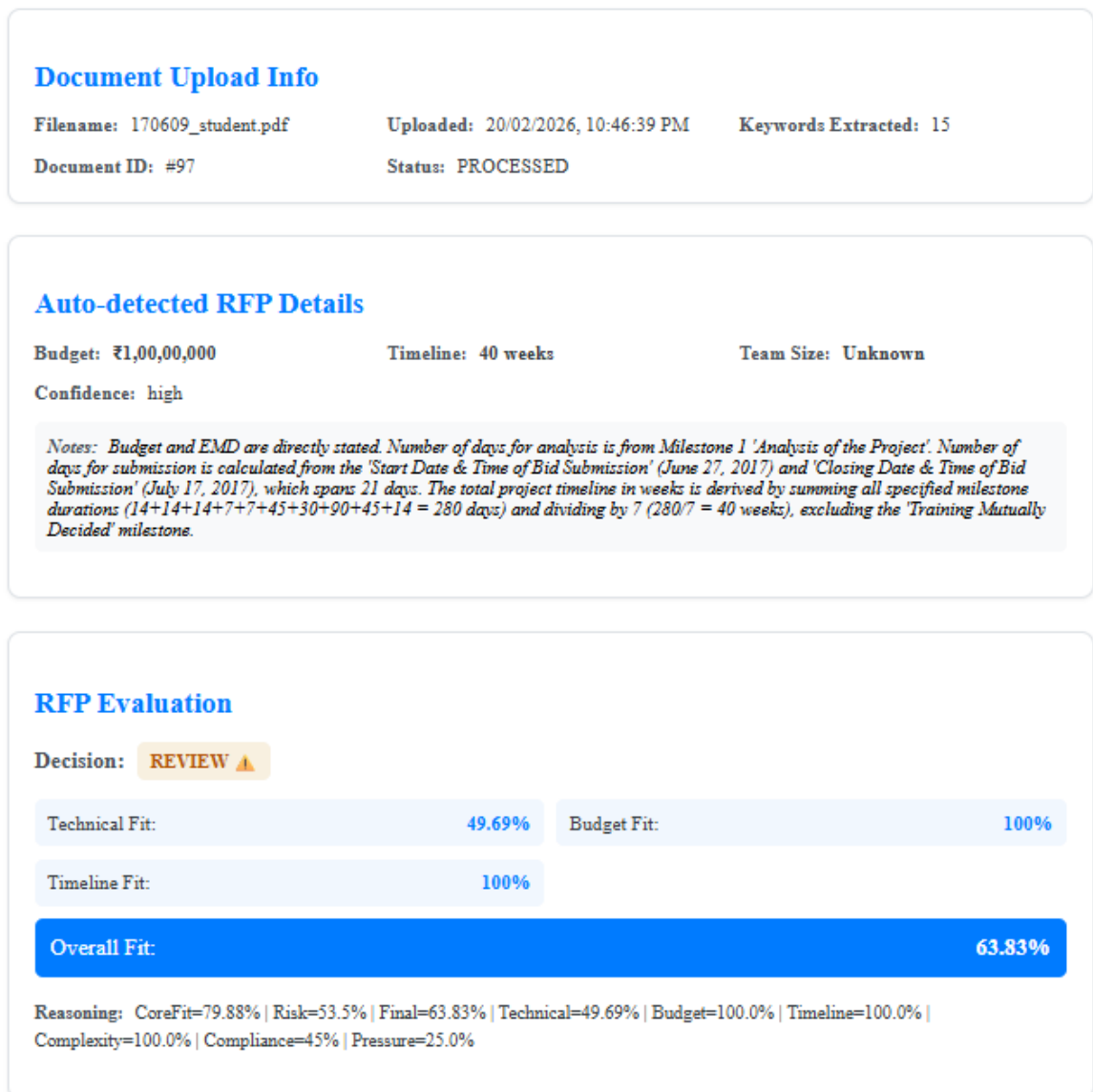


Figure 4.4: RFP Evaluation Results panel displaying animated progress bars for technical, budget, and timeline fit scores alongside the overall fit score and decision badge.

The technical fit score is computed through a four-layer keyword matching process. Layer one performs exact string matching between company keywords and document keywords. Layer two performs substring matching to handle partial phrase overlaps. Layer three searches the full raw document text for multi-word company keyword phrases. Layer four applies word overlap scoring, awarding a match when 50% or more of the words in a multi-word keyword phrase appear in the document. The raw keyword match score is then combined with a 30% semantic similarity boost computed by encoding all company keywords and all document keywords using all-MiniLM-L6-v2, averaging each set into a

single representative vector, and computing cosine similarity between the two vectors. The final technical fit score is a weighted blend of 70% keyword matching and 30% semantic similarity.

The budget fit score checks whether the extracted RFP budget falls within the company's configured minimum and maximum acceptable project value range, scoring 100% for budgets within range and scaling proportionally for those outside it. The timeline fit score applies the same range-based logic to the extracted project duration in weeks. The overall fit score is computed as a weighted average in which technical fit carries 50% weight and budget and timeline each carry 25%, reflecting the greater importance of technical capability over financial parameters in determining bid viability.

A score at or above 40% produces an ACCEPT decision, a score between 20% and 39% produces a REVIEW decision, and a score below 20% produces a REJECT decision. The evaluation panel displays a reasoning text block in plain language, identifying which keywords matched, which were absent, and whether the budget and timeline fell within the acceptable range, providing actionable transparency to the user regarding the basis of the evaluation outcome.

#### **4.7 Generate Proposal Banner**

When the evaluation module assigns an ACCEPT decision, a prominent green banner is rendered at the top of the results panel, visually distinguishing the document as a viable bid opportunity. The banner presents a Generate Proposal button alongside a brief message confirming bid viability. This design ensures that the proposal generation feature is surfaced only for documents that meet the minimum compatibility threshold, preventing unnecessary generation for poorly matched RFPs and directing user attention toward actionable opportunities.

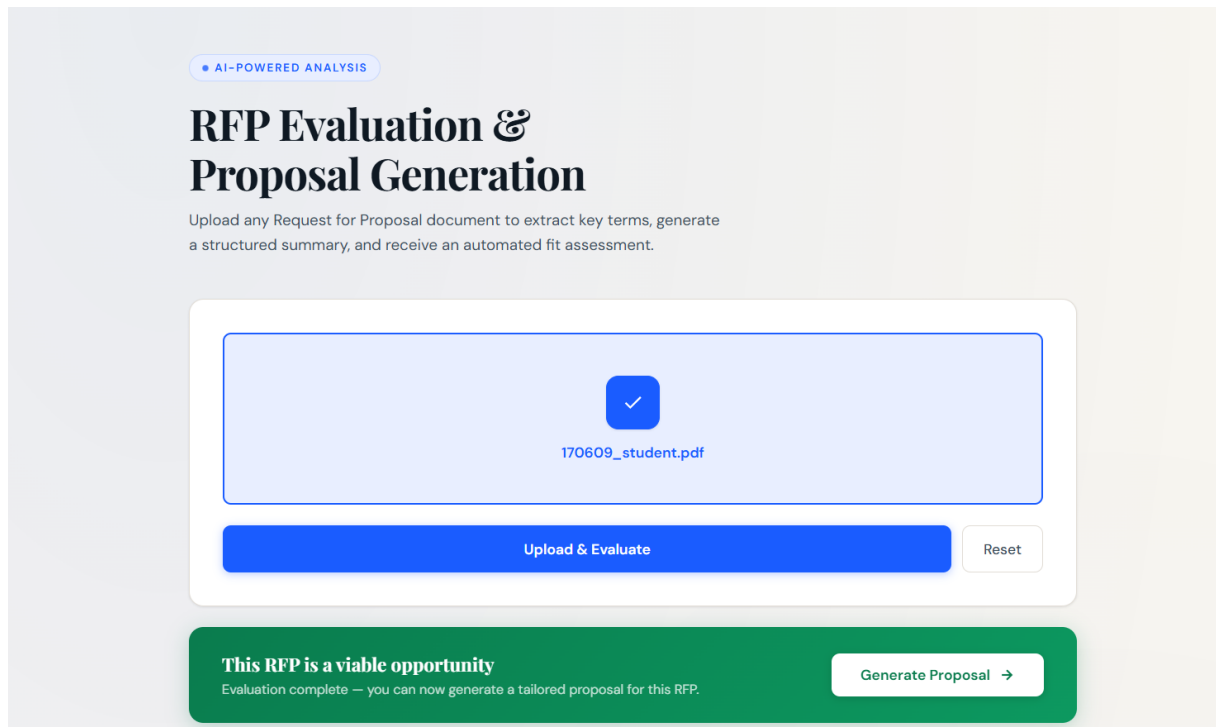


Figure 4.5: Upload Interface with ACCEPT Decision and Generate Proposal Banner confirming bid viability.

Clicking the Generate Proposal button navigates the user to the dedicated proposal generation page at `/proposal/{id}`, where the document ID is embedded in the URL to enable direct access and state restoration on return navigation.

## 4.8 Proposal Generation Results

The proposal generation page presents the AI-generated bid proposal produced by the Retrieval Augmented Generation pipeline. Upon navigating to the page, the system invokes the generation pipeline unless a cached proposal already exists in the database for the document, in which case the stored result is returned instantly without re-invoking the language model. During generation, an animated loading screen displays sequential progress steps — including RFP chunk retrieval, requirement analysis, and individual section generation — providing the user with continuous visibility into the pipeline status.

The RAG retrieval stage encodes a composite query covering scope, requirements, technical specifications, timeline, and compliance using all-MiniLM-L6-v2, and retrieves the four most semantically relevant document chunks from the embeddings stored in

PostgreSQL using cosine similarity. These chunks are concatenated with the document metadata — including extracted keywords, budget, and timeline — and passed as context to LLaMA 3.2 running locally via Ollama. Separate focused prompts are constructed for each of the six proposal sections, with a token generation limit of 600 per section and a temperature of 0.7 to balance coherence with natural variation in phrasing. All markdown formatting produced by the language model is stripped before display to ensure clean, readable output.

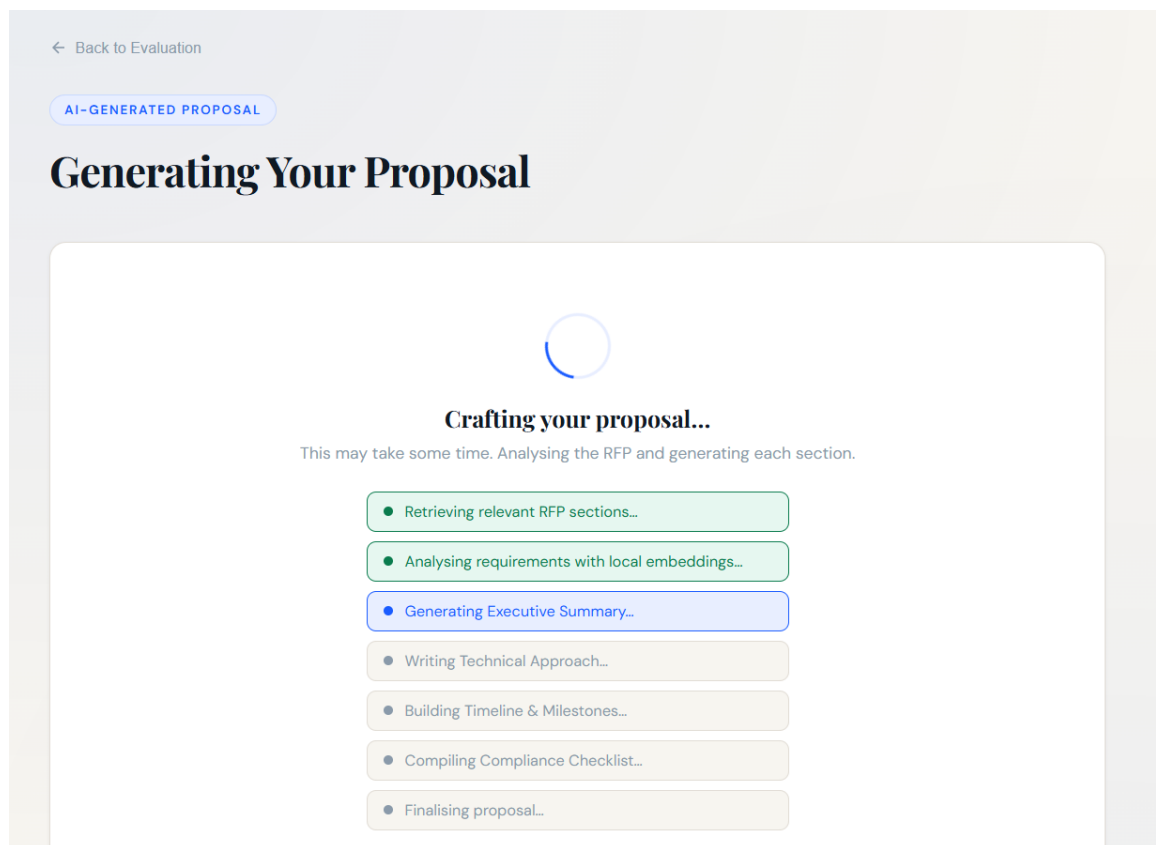


Figure 4.6: Proposal Generation Page showing the animated loading screen with sequential progress steps during LLaMA inference.



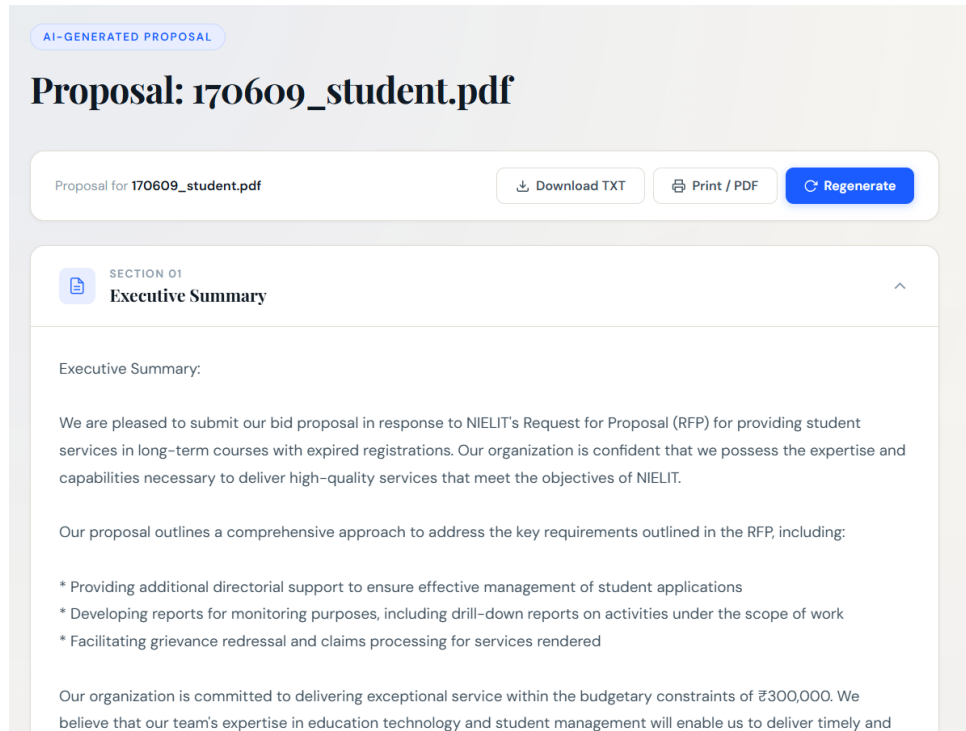


Figure 4.7: Generated Proposal Panel-a, showing Executive Summary, download, print and regenerate button.

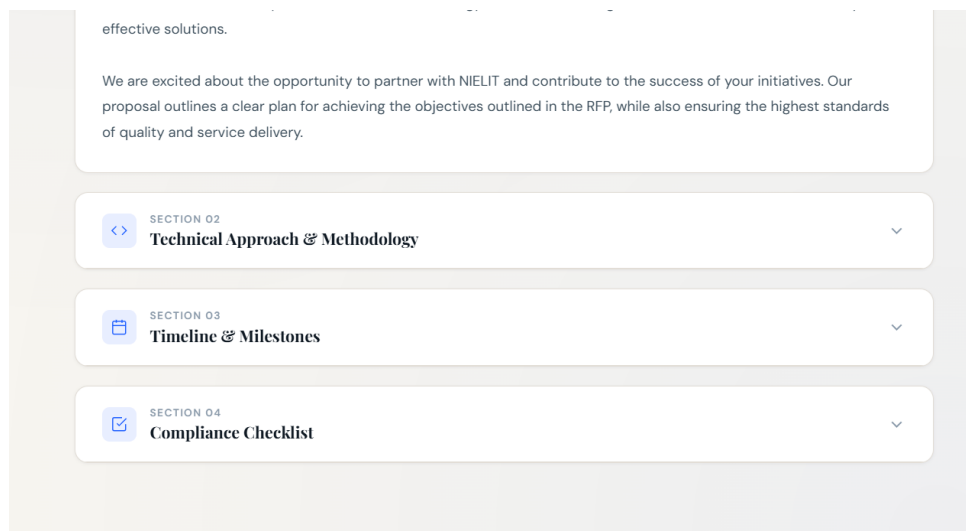


Figure 4.8: Generated Proposal Panel-b, showing Technical Approach, Timeline & Milestones, and Compliance Checklist sections.

### Technical Approach for NIELIT E-Bid Proposal

In response to the RFP (Request for Proposal) from NIELIT (National Institute of Electronics and Information Technology), we propose a customized e-bidding solution that leverages our expertise in technology stack, scalability, and security.

#### System Architecture

Our proposed system will be built using a modular architecture with the following components:

1. User Interface (UI): Built using UAT (User Acceptance Testing) framework, ensuring seamless integration with existing systems.
2. Backend Infrastructure: Cloud deployment for scalability, reliability, and high availability.
3. Authentication and Authorization: Implementing robust authentication mechanisms to ensure secure access to the system.
4. Data Management: Utilizing scalable data storage solutions (SRS) to manage large datasets.

#### Implementation Phases

Our approach will involve the following implementation phases:

1. System Design: Customizable solution tailored to meet NIELIT's specific requirements.
2. UAT and Testing: Comprehensive testing to ensure system meets NIELIT's standards.
3. Deployment: Cloud deployment for scalability and reliability.
4. Security Testing: Conducting thorough security testing to ensure the system meets NIELIT's security standards.

#### Security Measures

Our proposed solution will incorporate robust security measures, including:

1. Encryption: Implementing end-to-end encryption to protect sensitive data.
2. Access Control: Utilizing role-based access control to restrict user access.
3. Regular Security Audits: Conducting regular security audits to ensure compliance with NIELIT's security standards.

#### Scalability and Maintenance

Our solution will be designed to scale horizontally, ensuring that it can handle increasing volumes of data and users. Additionally, we will provide comprehensive maintenance and support services, including:

1. Regular Updates and Patches: Keeping the system up-to-date with latest security patches.
2. Performance Monitoring: Continuously monitoring system performance to ensure optimal efficiency.

#### Timeline

Our proposed solution is expected to be completed within 12 weeks, with the following milestones:

- \* Week 4: System Design and UAT
- \* Week 8: Testing and Security Audits
- \* Week 10: Deployment and Training

We believe our technical approach meets NIELIT's requirements and look forward to the opportunity to collaborate on this project.

Figure 4.9: Generated Proposal Panel-c, showing Technical Approach section.



## SECTION 03

### Timeline & Milestones

Based on the provided RFP and a standard project timeline framework, I've created a 25-week project timeline for your reference:

#### Phase 1: Initiation (Weeks 1-3)

\* Week 1:

+ Milestone: Project Kickoff Meeting – Introduce project team, stakeholders, and objectives.

\* Week 2:

+ Milestone: Requirements Gathering – Collect and document project requirements from stakeholders.

\* Week 3:

+ Milestone: Project Scope Definition – Finalize project scope, goals, and deliverables.

#### Phase 2: Design (Weeks 4-8)

\* Week 4:

+ Milestone: High-Level Design Concept – Present initial design concepts to stakeholders.

\* Week 5:

+ Milestone: Detailed Design Requirements – Document and finalize design requirements.

\* Week 6:

+ Milestone: Wireframe Development – Create low-fidelity wireframes for user interface (UI) and user experience (UX).

\* Week 7:

+ Milestone: Prototype Development – Develop a functional prototype for UI/UX testing.

\* Week 8:

+ Milestone: Design Review – Conduct design review with stakeholders to finalize design.

#### Phase 3: Development (Weeks 9-18)

\* Week 9:

+ Milestone: Coding Begins – Start coding the project based on approved design requirements.

\* Week 10:

+ Milestone: Unit Testing – Begin unit testing individual components and modules.

\* Week 11:

+ Milestone: Integration Testing – Integrate developed components and modules into a cohesive system.

\* Week 12:

+ Milestone: Functional Testing – Perform functional testing to validate the project's core functionality.

\* Week 13:

+ Milestone: Performance Testing – Conduct performance testing to ensure the project meets performance expectations.

\* Week 14:

+ Milestone: Security Auditing – Perform security auditing and vulnerability testing.

\* Week 15:

+ Milestone: Quality Assurance Review – Conduct a thorough review of the project's quality and stability.

\* Week 16-17:

+ Milestone: Code Refactoring – Refactor code to improve maintainability, readability, and performance.

\* Week 18:

+ Milestone: Finalize Development – Complete all development tasks and prepare for testing.

Figure 4.10: Generated Proposal Panel-d, showing Timeline & Milestones section.

Here is a comprehensive compliance checklist for the RFP bid proposal:

#### Technical Compliance

- ✓ UAT (User Acceptance Testing) – We will conduct thorough UAT of our system to ensure it meets the requirements and is ready for deployment.
- ✓ Cloud Deployment – Our system will be deployed on a scalable cloud infrastructure, ensuring high availability and reliability across PAN India.
- ✓ Customizable System – Our system will be customizable to meet the specific needs of NIELIT, including authentication and authorization features.
- ✓ Authentication – We will implement robust authentication mechanisms to ensure secure access to our system.
- ✓ Testing – Our system will undergo rigorous testing, including functional, performance, and security testing, to ensure it meets the requirements.
- ✓ Maintenance – We will provide regular maintenance and support services to ensure the system remains up-to-date and secure.
- ✓ SRS (System Requirements Specification) – Our system will adhere to the provided SRS document, ensuring all requirements are met.

#### Eligibility Compliance

- ✓ NIELIT Invites E-Bids – We have reviewed and understood the eligibility criteria set by NIELIT for e-bid proposals.
- ✓ Portal Registration – We will register our organization on the NIELIT portal as required, ensuring timely submission of bids.
- ✓ Additional Director NIELIT – Our proposal includes the required information about our additional director, as per NIELIT's guidelines.

#### Financial Compliance

- ✓ Bid Validity Days – We have calculated and included the bid validity days in our proposal, as specified by NIELIT.
- ✓ Technical Bid – Our technical bid includes all required documents and information, meeting the financial eligibility criteria set by NIELIT.

#### Legal Compliance

- ✓ Bid Cert-In Indian – Our proposal includes the required certificate from a registered certification body, as per NIELIT's guidelines.
- ✓ Main Objectives NIELIT – Our proposal aligns with the main objectives of NIELT, including providing quality education and training to students.
- ✓ Registration on NIELIT Autonomous Scientific – We will register our organization on the NIELIT Autonomous Scientific portal, ensuring compliance with all requirements.
- ✓ Inc. NIELT National – Our proposal includes the required information about our organization's affiliation with NIET and NIN, as per NIET's guidelines.

Figure 4.11: Generated Proposal Panel-e, showing Compliance Checklist section.

The Executive Summary introduces the proposal in line with RFP requirements, states the organisation’s intent, and highlights technical capabilities from the company profile. The Technical Approach outlines a phased methodology referencing relevant technologies and architectural patterns. The Timeline presents a structured plan covering initiation, requirements gathering, design, development, testing, deployment, and handover, scaled to the RFP’s timeline. The Budget Breakdown provides component-wise cost estimates for development, infrastructure, testing, management, training, and contingency, proportioned to the RFP’s project value. The Team Structure proposes defined roles and responsibilities tailored to the scope. The Compliance Checklist generates itemised confirmations across technical, eligibility, financial, and legal categories, each paired with a compliance statement grounded in the RFP.

#### **4.9 Summary of Chapter**

This chapter presented the end-to-end results of the Automated Proposal Generation System evaluated on a real government RFP document. The document processing pipeline completed within 30 to 60 seconds, producing 15 semantically ranked keywords, a structured hybrid summary correctly identifying the issuer, purpose, project value, EMD, and bid validity, and metadata extracted with a confidence level reflecting the completeness of structured information available in the source document. The evaluation module computed fit scores across technical, budget, and timeline dimensions, producing a decision that determined whether the proposal generation feature was enabled. The RAG-based proposal generation pipeline produced a complete six-section bid proposal using LLaMA 3.2 via Ollama, with each section grounded in the actual RFP content retrieved through cosine similarity over stored MiniLM embeddings. The results demonstrate that the system successfully fulfils its core objective of automating the RFP analysis and proposal generation pipeline using entirely local, open-source models without dependence on any external cloud API, ensuring complete data privacy and zero inference cost.

## Chapter 5

### Conclusion

#### 5.1 System Implementation Overview

This project successfully designed and implemented an end-to-end AI-powered Automated Proposal Generation System capable of processing Request for Proposal (RFP) documents and producing structured bid proposals with minimal human intervention. The system integrates a multi-stage pipeline combining rule-based extraction, local transformer models, and a locally hosted large language model to deliver a fully offline, privacy-preserving solution.

The document processing module supports PDF, DOCX, and TXT formats using PyMuPDF and python-docx, extracting raw text and splitting it into overlapping chunks for downstream retrieval. Keyword extraction was implemented using a custom YAKE-inspired statistical algorithm combined with semantic reranking using the all-MiniLM-L6-v2 sentence transformer, producing contextually meaningful keywords without requiring any external API or labelled training data. Document summarization was achieved through a hybrid approach combining regex-based extraction of structured facts such as issuer, purpose, budget, EMD, deadline, and bid validity, with an Ollama-generated scope sentence for improved accuracy.

The RFP metadata extraction module uses regex patterns to identify budget, timeline, team size, and EMD values, assigning a confidence level based on extraction completeness. The competency evaluation module computes three fit scores, technical, budget, and timeline, using a four-layer keyword matching algorithm augmented by a semantic similarity boost from all-MiniLM-L6-v2 embeddings, producing an overall fit score and an ACCEPT, REVIEW, or REJECT decision. The company capability profile is managed through the Django admin interface, allowing administrators to configure acceptable budget ranges, timeline ranges, and a comprehensive list of technical keywords.

The proposal generation module implements a Retrieval Augmented Generation (RAG) pipeline in which the most relevant RFP chunks are retrieved using cosine similarity over stored MiniLM embeddings, and passed as context to LLaMA 3.2 running locally via Ollama to generate six structured proposal sections: Executive Summary, Technical Approach and Methodology, Timeline and Milestones, Budget Breakdown, Proposed Team Structure, and Compliance Checklist. The Gemini 2.0 Flash API was integrated in parallel as a comparative generation backend, enabling a study of local versus cloud-based generation in terms of output quality, cost, privacy, and response time.

The frontend was built using Next.js with the app router, featuring a professional editorial interface with animated evaluation score bars, collapsible proposal sections, TXT download, print to PDF, and URL-based state restoration ensuring evaluation results persist across page navigation. The backend was implemented using Django REST Framework with PostgreSQL, exposing six REST API endpoints for document upload, retrieval, evaluation, and proposal generation.

## **5.2 Summary of Chapter**

This chapter presented the end-to-end results of the Automated Proposal Generation System evaluated on a real government RFP document. The document processing pipeline completed within 30 to 60 seconds, producing 15 semantically ranked keywords, a structured hybrid summary correctly identifying the issuer, purpose, project value, EMD, and bid validity, and metadata extracted with confidence reflecting the completeness of structured information available. The evaluation module computed fit scores across technical, budget, and timeline dimensions, producing a decision that determined whether the proposal generation feature was enabled. The RAG-based pipeline produced a complete six-section bid proposal using LLaMA 3.2 via Ollama, with each section grounded in the actual RFP content retrieved through cosine similarity over stored MiniLM embeddings. The results demonstrate that the system fulfils its core objective of automating RFP analysis and proposal generation using entirely local, open-source models without dependence on external cloud APIs, ensuring complete data privacy and zero inference cost.

## List of Publications

1. “Retrieval-Augmented Generation Based Framework for Automated RFP Response Generation,” (*Under Review*), Submitted to: SCMS School of Engineering and Technology,” Fourth International Conference on Computing, Communication, Security & Intelligent Systems (IC3SIS 2026)”, Available: <https://www.ic3sis.in/>, Accessed: March 2026.



## References

- [1] A. J. Yepes, Y. You, J. Milczek, S. Laverde, and L. Li, “Financial report chunking for effective retrieval augmented generation,” *arXiv preprint arXiv:2402.05131*, 2024.
- [2] A. Smadi, A. Al-Qerem, A. Nabot, I. Jebreen, A. Aldweesh, M. Alauthman, and Others, “Unlocking the potential of competency exam data with machine learning: Improving higher education evaluation,” *Sustainability*, vol. 15, no. 5267, 2023.
- [3] H. Guo, S. Zhang, S. Narayan, and G. Durrett, “Questioning the validity of summarization datasets and improving their factual consistency,” *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1135–1153, 2022.
- [4] P. Nistala, A. Rajbhoj, V. Kulkarni, S. Noronha, and A. Joshi, “An industrial experience report on model-based, ai-enabled proposal development for an rfp/rfi,” *Science of Computer Programming*, vol. 264, p. 103058, 2024.
- [5] S. Gupta, A. Verma, and D. Menon, “Automating request for proposal (rfp) analysis using natural language processing and deep learning,” *Procedia Computer Science*, vol. 220, pp. 501–509, 2023.
- [6] A. Raman and P. Deshmukh, “Generative ai in enterprise automation: Opportunities and challenges,” *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 5, pp. 88–95, 2022.
- [7] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [8] M. Bhatia, S. Jain, and S. Sood, “Artificial intelligence-based proposal management system using nlp and knowledge mapping,” *Journal of Intelligent Systems*, vol. 31, no. 1, pp. 123–136, 2022.

- [9] S. Choudhury and P. Tiwari, “Leveraging ai to automate request for proposal (rfp) responses,” *International Journal of Computer Applications*, vol. 183, no. 10, pp. 25–30, 2021.
- [10] Google Cloud, “Using ai for document understanding and automation,” *Google Cloud Technical Report*, pp. 1–5, 2023.
- [11] Microsoft Research, “Automating business proposals with large language models,” *Microsoft Research Technical Report*, pp. 1–6, 2023.

# Appendix A: Presentation

# Automated Proposal Generation based on Request for Proposal (RFP)

**Final Presentation**

Guide : Ms. Sangeetha Jamal  
Asst. Professor, DCS, RSET

Rahul Rathishkumar Menon (U2203173)  
Niranjana N (U2203163)  
P.A Abidh Muhammed (U2203171)  
Nevin John George (U2203154)  
S8 - CS Gamma

**In collaboration with**

**10<sup>x</sup> DS<sup>TM</sup>**

**Exponential Digital Solutions Pvt.**

# TABLE OF CONTENTS

|   |                     |
|---|---------------------|
| 1 | Introduction        |
| 2 | Problem Definition  |
| 3 | Problem Statement   |
| 4 | Purpose & Need      |
| 5 | Literature Survery  |
| 6 | Achitecture Diagram |
| 7 | Module Wise Diagram |

|    |                                  |
|----|----------------------------------|
| 8  | Result and Output                |
| 9  | Hardware & Software Requirements |
| 10 | Gantt Chart                      |
| 11 | Risk and Challenges              |
| 12 | Conclusion                       |
| 13 | References                       |

3

## Introduction

- Business organizations and governments use RFPs to invite bids for products or services.
- Responding to RFPs involves handling many detailed questions and requirements.
- Presales teams currently interpret questions manually and compile responses.
- The proposed system automates proposal creation using templates and learns from previous proposals.
- This approach reduces manual work, shortens response times, and delivers more accurate, consistent proposals.

4



## Problem Definition

Organizations often invest significant amounts of time and expert resources in reviewing RFPs, identifying compliance requirements, and preparing proposal drafts. Therefore, there is a pressing need for an intelligent, automated system that can streamline and standardize the proposal creation process while reducing manual dependency.

5



## Problem Statement

To develop an AI-driven RFP Evaluation and Proposal Generation framework that analyzes and classifies Request for Proposals using semantic retrieval, structured scoring, and Retrieval-Augmented Generation with a locally deployed Large Language Model.



6

# Purpose and Need

The primary purpose of the AI-Based RFP Evaluation and Proposal Generation System is to automate and support decision-making in the Request for Proposal (RFP) lifecycle. Organizations receive numerous RFPs, each requiring detailed analysis to determine feasibility, technical alignment, budget suitability, and timeline compatibility. Traditionally, this evaluation process is manual, time-consuming, and heavily dependent on subject matter experts (SMEs). The proposed system aims to reduce this dependency by implementing an intelligent, data-driven, and explainable decision support mechanism that can automatically analyze RFP documents, extract structured metadata, evaluate organizational compatibility, and classify the RFP as ACCEPT, REJECT, or REVIEW.

7

# Purpose and Need

The need for this system arises from several critical limitations in current RFP handling practices.

- 1) Companies often receive a large volume of RFPs, making manual screening inefficient and prone to inconsistency.
- 2) Purely generative AI tools often produce hallucinated or unverified content.
- 3) Many AI-based proposal tools rely on cloud APIs, raising confidentiality concerns.

8

# Literature Survey

## **“Model-based, AI-enabled Proposal development for an RFP/RFI”**

P. Nistala, A. Rajbhoj, V. Kulkarni, S. Noronha, and A. Joshi, “An industrial experience report on model-based, AI-enabled proposal development for an RFP/RFI,” [Journal/Conference Name], Elsevier, 2024. [1]

- This paper proposes a Model-Driven Engineering (MDE) based approach for automating proposal development.
- A Document-to-Model (DocToModels) engine parses unstructured documents and populates the knowledge model.
- DocToModels automatically reads solution knowledge and populates the Proposal Knowledge Model.
- A knowledge-map recommender retrieves relevant information from the model, and a Model-to-Text transformation generates the proposal document.
- Fetches matching KnowledgeElements and presents them as recommendations

9

# Literature Survey

## **“An RFP System for Generating Response to a Request for Proposal”**

A. Rajbhoj, P. Nistala, V. Kulkarni, and G. Ganesan, “An RFP system for generating response to a request for proposal,” in Proceedings of the ACM Innovations in Software Engineering Conference (ISEC), Pune, India, 2019, pp. —, doi: 10.1145/3299771.3299779. [2]

- Study presents a knowledge-driven and model-based RFP response generation system.
- The system uses NLP techniques to analyze RFP questions and generate search queries to retrieve relevant answers from a knowledge repository.
- Retrieved content is ranked and composed into a structured draft response.

10



# Literature Survey

## **“Generative AI for Sales: RAG for RFP Automation”**

[3] E. Bilal and O. Åström, “Generative AI for sales: A study on the potential of retrieval augmented generation for response automation in the request for proposal process in telecommunications,” Degree Project, KTH Royal Institute of Technology, Stockholm, Sweden, 2024

- This paper implements a RAG framework to automate RFP response creation in the telecommunications domain.
- The system consists of three components: a retriever, generator, and pre-retrieval processing layer.
- Various chunking strategies (small chunks, large chunks, and parent-child chunking) are evaluated to determine optimal performance.
- The retriever extracts relevant context from a domain-specific knowledge base, and a generative model produces responses grounded in retrieved information.

11

# Literature Survey

## **“Retrieval-Augmented Multi-Agent System for Rapid SOW Generation”**

[4] A. Suravarjhula, R. C. Agrawal, S. J. Patel, and R. Gupta, “Retrieval-augmented multi-agent system for rapid statement of work generation,” arXiv preprint arXiv:2508.07569, 2025.

- This study introduces a multi-agent Retrieval-Augmented Generation system for automating Statement of Work drafting.
- It consists of three coordinated agents: a Drafting Agent, Compliance Agent, and a Formatting and Validation Agent that ensures structural integrity.
- The system uses RAG to retrieve relevant clauses from a database, grounding generation in factual content and reducing hallucination. Agents communicate sequentially to refine the document and produce a legally compliant final output.

12

# Literature Survery

## “Questioning the Validity of Summarization Datasets”

Y. Guo, C. Clavel, M. Kamal Eddine, and M. Vazirgiannis, “Questioning the validity of summarization datasets and improving their factual consistency,” arXiv preprint arXiv:2210.17378, 2022. [5]

- This paper evaluates the factual consistency of popular abstractive summarization datasets using factual consistency models.
- combines entailment classification and question-answering based evaluation models to detect inconsistencies between summaries and source documents.
- Based on these evaluations, the authors construct a filtered dataset (SummFC) that improves factual reliability. The study demonstrates that training models on factually consistent datasets improves performance across multiple quality metrics.

13

# Literature Summary

| Category               | GenAI for Sales(Bilal & Åström, KTH 2024)                          | Questioning Summarization Datasets(Guo et al., 2022)                      | RAG Multi-Agent SOW Generation(Suravarjhu a et al., 2025)              | Model-Based AI Proposal Dev.(Nistala et al., Elsevier 2024)                            |
|------------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Core Problem Addressed | RFP responses require domain-specific, accurate content generation | Summarization datasets contain factually inconsistent reference summaries | Manual SOW drafting is slow, error-prone, and resource-intensive       | Manual, SME-dependent proposal creation is slow and inconsistent across large BUs      |
| Key Technology Stack   | LangChain, PyMuPDF, bge-small-en embeddings, LLaMA2                | BART fine-tuning, BERTScore, BARTScore                                    | GPT-4.1 (drafting), BART (compliance), PostgreSQL, LangGraph, Flask    | MOF modeling Docx4J, ML classifiers                                                    |
| Evaluation Method      | Automatic + human evaluation.                                      | Automatic metrics: BERTScore, BARTScore, statistical significance         | Checklist-based review (96% accuracy),20 legal professional evaluators | User productivity data (time before/after); qualitative user feedback; 700+ real users |

14

| Category                  | GenAI for Sales(Bilal & Åström, KTH 2024)                                                                                 | Questioning Summarization Datasets(Guo et al., 2022)                                                            | RAG Multi-Agent SOW Generation(Suravarjhula et al., 2025)                                                                             | Model-Based AI Proposal Dev.(Nistala et al., Elsevier 2024)                                                                         |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Key Results / Performance | Parent-child chunking 0.750 (best); advanced questions need larger chunks; faithfulness high; answer correctness moderate | models trained on SummFC outperform on nearly all quality metrics incl. factual consistency and informativeness | Draft in under 3 min (vs. 5–7 days manually); 96% accuracy; 81% legal accuracy; 71% writing similarity                                | 50–70% of proposal questions auto-answered; turnaround reduced from 24–48 hrs to 2–3 hrs; 70–75% effort automated                   |
| Limitations / Challenges  | Poor table parsing; simple queries may miss partial information.                                                          | Limited to English, no human evaluation of filtered summaries                                                   | Human oversight still needed for edge cases; reliance on OpenAI API raises data privacy concerns; no multi-language support mentioned | Initial knowledge population is effort-intensive; NLP handling of taxonomical variation; scaling to more domains takes 12–18 months |
| Automation Level          | <b>Semi-automated:</b> RAG generates first draft; human expert refines                                                    | <b>Fully automated</b> filtering pipeline; human-in-loop for final benchmark validation                         | <b>Highly automated:</b> end-to-end draft generation with auto-validation; human oversight for exceptions                             | <b>Highly automated:</b> 50–70% of questions answered; human SME selects and refines K-Map recommendations                          |

# Proposed Method

The system follows a eight-stage processing pipeline:

- Document Parsing & Preprocessing
- Semantic Chunking with Overlap
- Embedding Generation using Transformer-based Models
- Custom Vector Storage in PostgreSQL
- Context-Aware Retrieval using Cosine Similarity
- Multi-Factor Scoring & Decision Engine
- Proposal Generation
- Downloadable Proposal Document

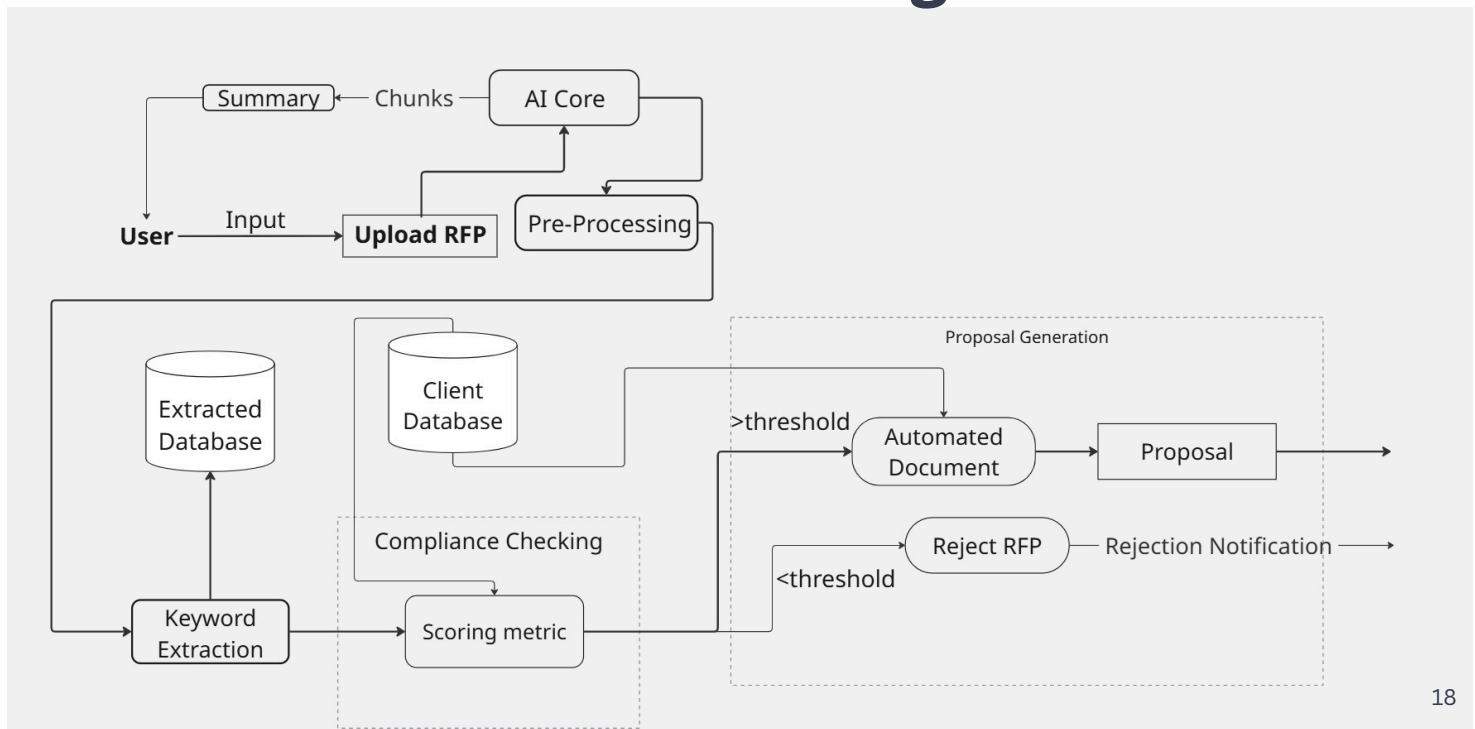
# Proposed Method

## The methodology ensures:

- Fully offline operation
- Deterministic decision-making
- Explainable scoring
- Context-grounded proposal drafting

17

## Architecture Diagram



18

# Module wise Diagrams

19

## Input/Output Module

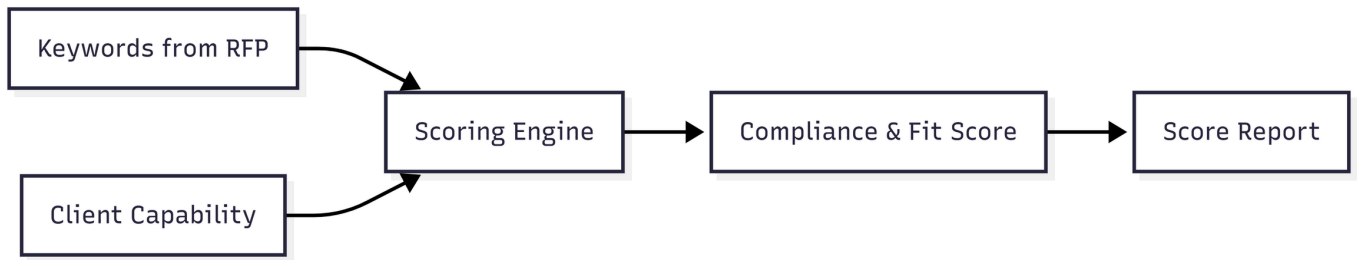


## Keyword Extraction Module

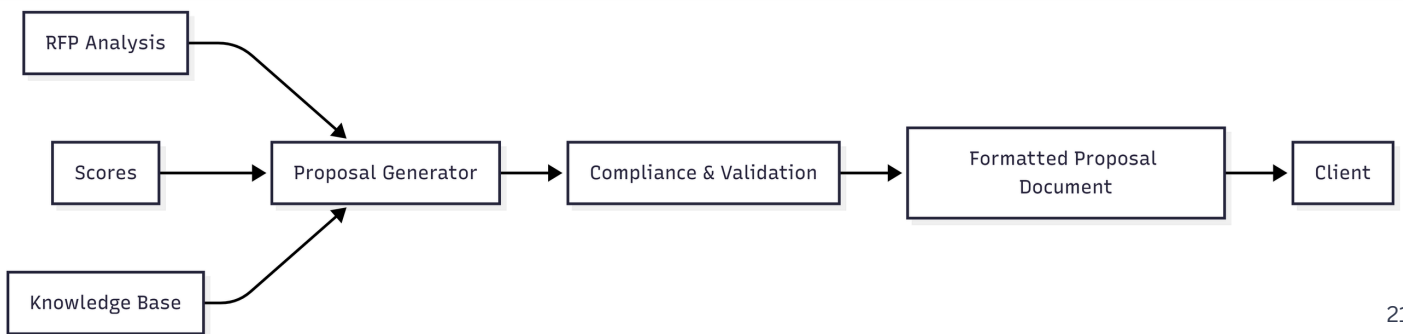


20

## Scoring Module

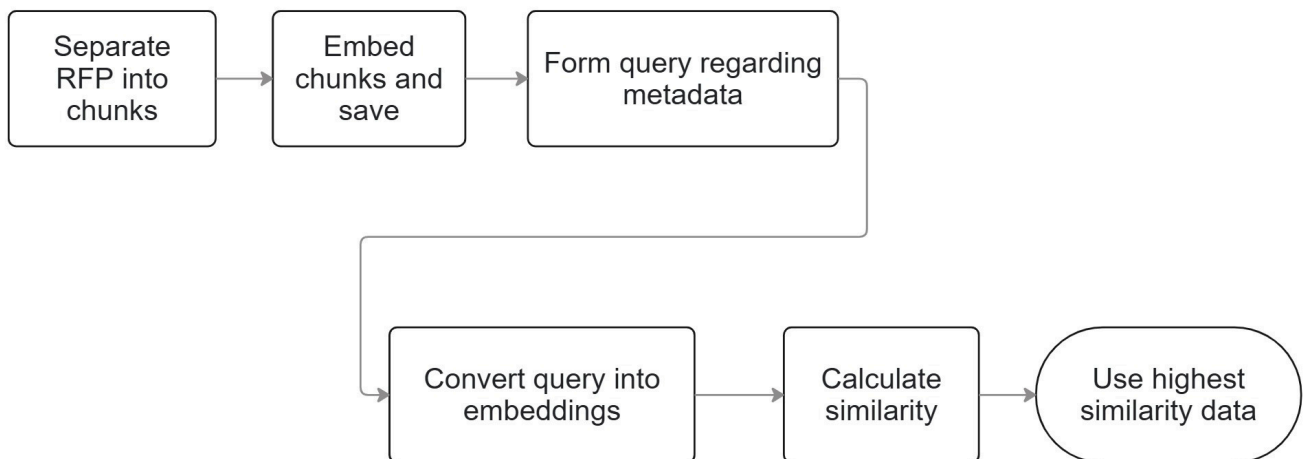


## Proposal Generation Module



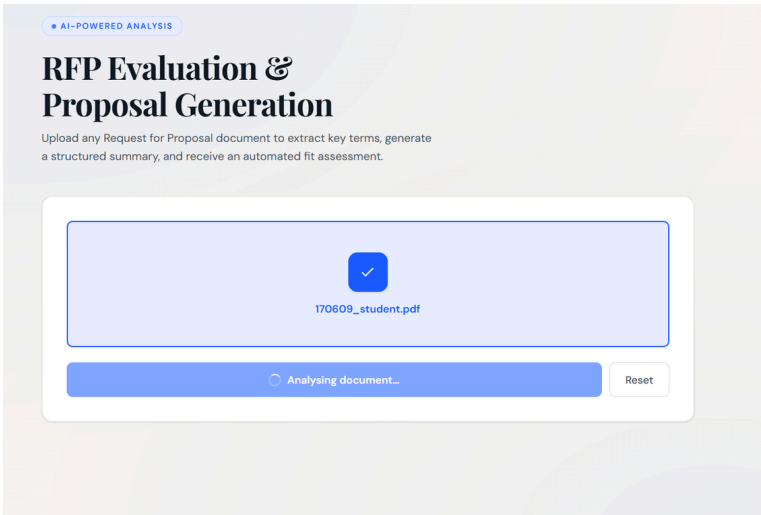
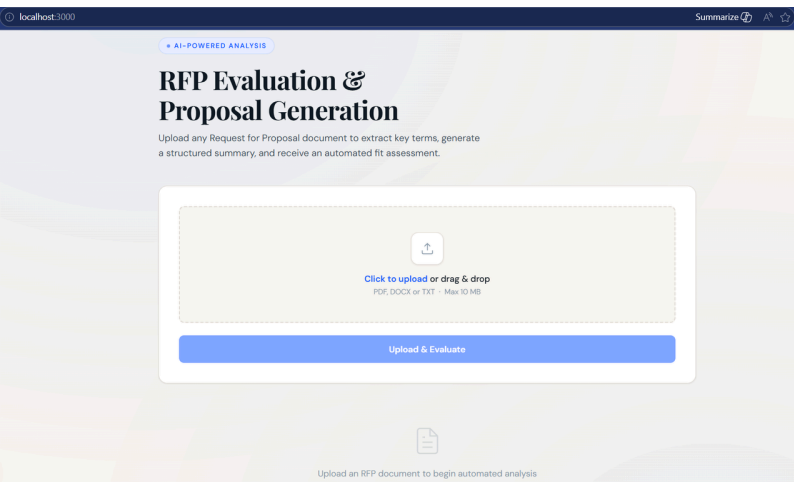
21

## RAG Module

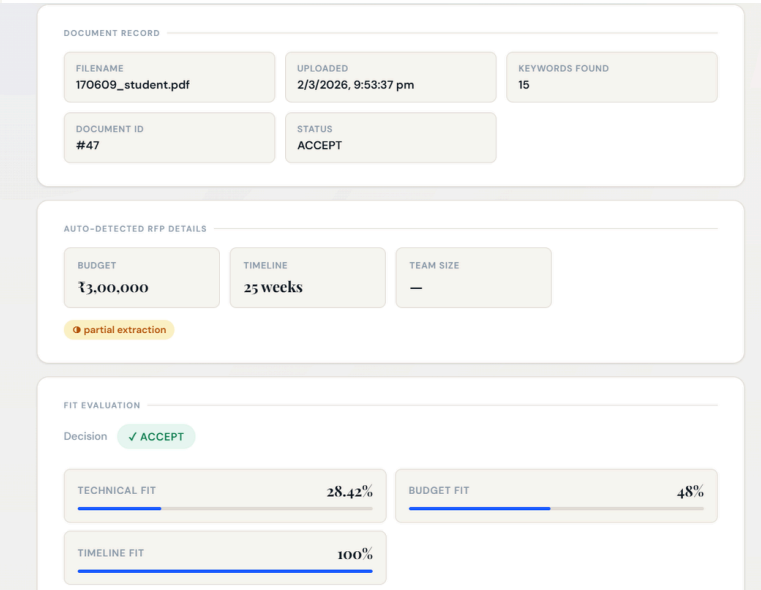
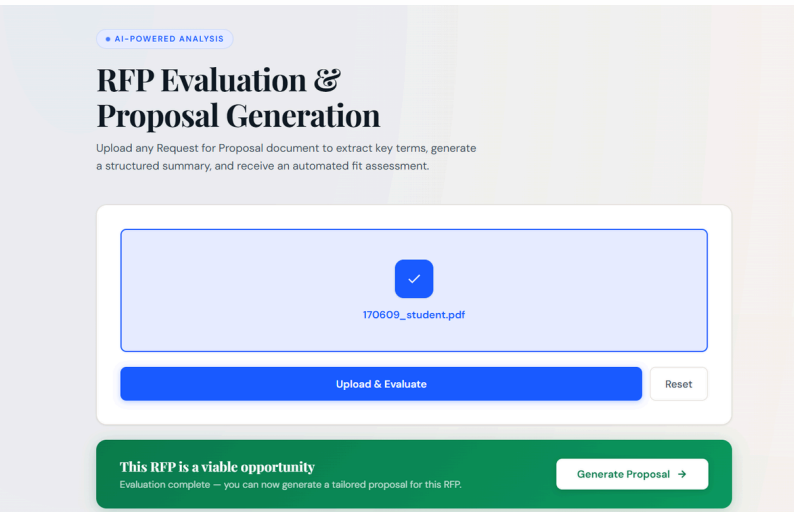


22

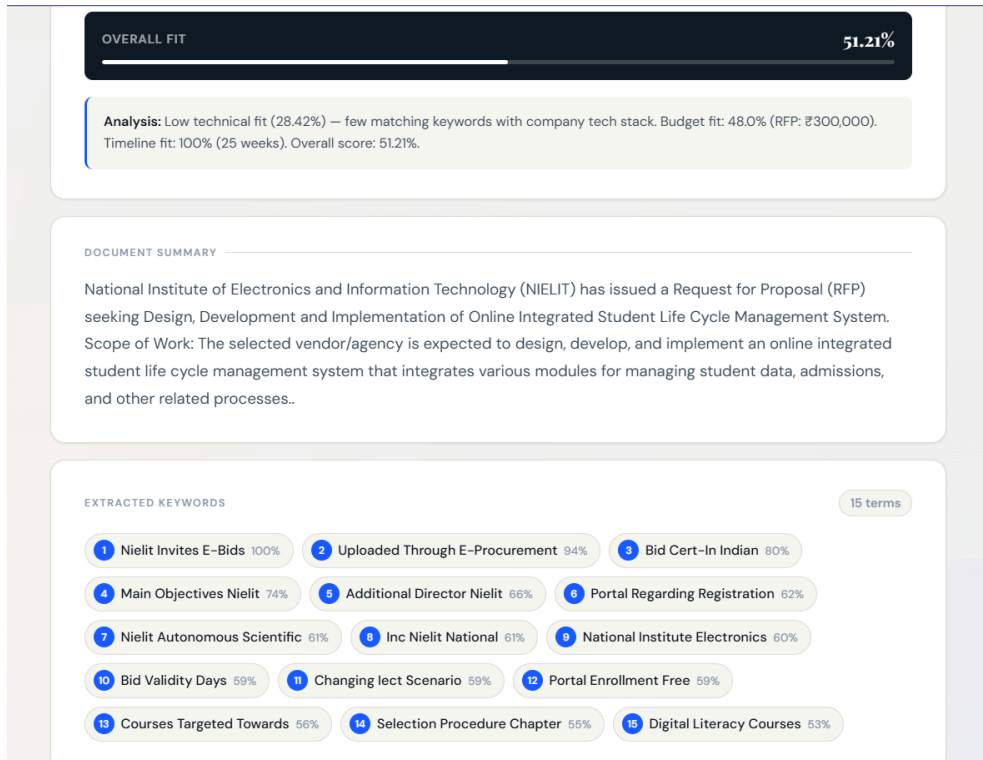
# Result and Outputs



23



24



← Back to Evaluation

AI-GENERATED PROPOSAL

## Generating Your Proposal

### Crafting your proposal...

This may take some time. Analysing the RFP and generating each section.

Retrieving relevant RFP sections...

Analysing requirements with local embeddings...

Generating Executive Summary...

Writing Technical Approach...

Building Timeline & Milestones...

Compiling Compliance Checklist...

Finalising proposal...

AI-GENERATED PROPOSAL

## Proposal: 170609\_student.pdf

Proposal for 170609\_student.pdf

Download TXT

Print / PDF

Regenerate

SECTION 01

Executive Summary

Executive Summary:

We are pleased to submit our bid proposal in response to NIELIT's Request for Proposal (RFP) for providing student services in long-term courses with expired registrations. Our organization is confident that we possess the expertise and capabilities necessary to deliver high-quality services that meet the objectives of NIELIT.

Our proposal outlines a comprehensive approach to address the key requirements outlined in the RFP, including:

- \* Providing additional directorial support to ensure effective management of student applications
- \* Developing reports for monitoring purposes, including drill-down reports on activities under the scope of work
- \* Facilitating grievance redressal and claims processing for services rendered

Our organization is committed to delivering exceptional service within the budgetary constraints of ₹300,000. We believe that our team's expertise in education technology and student management will enable us to deliver timely and



effective solutions.

We are excited about the opportunity to partner with NIELIT and contribute to the success of your initiatives. Our proposal outlines a clear plan for achieving the objectives outlined in the RFP, while also ensuring the highest standards of quality and service delivery.



SECTION 02

## Technical Approach & Methodology



SECTION 03

## Timeline & Milestones



SECTION 04

## Compliance Checklist



SECTION 02

## Technical Approach & Methodology



### Technical Approach for NIELIT E-Bid Proposal

In response to the RFP (Request for Proposal) from NIELIT (National Institute of Electronics and Information Technology), we propose a customized e-bidding solution that leverages our expertise in technology stack, scalability, and security.

#### System Architecture

Our proposed system will be built using a modular architecture with the following components:

1. User Interface (UI): Built using UAT (User Acceptance Testing) framework, ensuring seamless integration with existing systems.
2. Backend Infrastructure: Cloud deployment for scalability, reliability, and high availability.
3. Authentication and Authorization: Implementing robust authentication mechanisms to ensure secure access to the system.
4. Data Management: Utilizing scalable data storage solutions (SRS) to manage large datasets.

#### Implementation Phases

Our approach will involve the following implementation phases:

1. System Design: Customizable solution tailored to meet NIELIT's specific requirements.
2. UAT and Testing: Comprehensive testing to ensure system meets NIELIT's standards.
3. Deployment: Cloud deployment for scalability and reliability.

4. Security Testing: Conducting thorough security testing to ensure the system meets NIELIT's security standards.

#### Security Measures

Our proposed solution will incorporate robust security measures, including:

1. Encryption: Implementing end-to-end encryption to protect sensitive data.
2. Access Control: Utilizing role-based access control to restrict user access.
3. Regular Security Audits: Conducting regular security audits to ensure compliance with NIELIT's security standards.

#### Scalability and Maintenance

Our solution will be designed to scale horizontally, ensuring that it can handle increasing volumes of data and users. Additionally, we will provide comprehensive maintenance and support services, including:

1. Regular Updates and Patches: Keeping the system up-to-date with latest security patches.
2. Performance Monitoring: Continuously monitoring system performance to ensure optimal efficiency.

#### Timeline

Our proposed solution is expected to be completed within 12 weeks, with the following milestones:

- \* Week 4: System Design and UAT
- \* Week 8: Testing and Security Audits
- \* Week 10: Deployment and Training

We believe our technical approach meets NIELIT's requirements and look forward to the opportunity to collaborate on

Selected: 170609\_student.pdf

Upload & Evaluate

Reset

Document Upload Info

Filename: 170609\_student.pdf

Uploaded: 20/02/2026, 10:46:39 PM

Keywords Extracted: 15

Document ID: #97

Status: PROCESSED

Auto-detected RFP Details

Budget: ₹1,00,00,000

Timeline: 40 weeks

Team Size: Unknown

Confidence: high

Notes: Budget and EMD are directly stated. Number of days for analysis is from Milestone 1 'Analysis of the Project'. Number of days for submission is calculated from the 'Start Date & Time of Bid Submission' (June 27, 2017) and 'Closing Date & Time of Bid Submission' (July 17, 2017), which spans 21 days. The total project timeline in weeks is derived by summing all specified milestone durations (14+14+14+7+7+45+30+90+45+14 = 280 days) and dividing by 7 (280/7 = 40 weeks), excluding the 'Training Manually Decided' milestone.

RFP Evaluation

Decision: REVIEW ▲

Technical Fit: 49.69%

Budget Fit: 100%

Timeline Fit: 100%

Overall Fit: 63.83%

29

Document Summary

The National Institute of Electronics and Information Technology (NIELIT), under the Ministry of Electronics and IT (MeitY), Government of India, issued a Request for Proposal (RFP) on June 10, 2017. This RFP seeks a competent and professional bidder for the "Design, Development, and Implementation of an Online Integrated Student Life Cycle Management System".

**Main Objectives:** The overarching objective is to establish a scalable, secure, robust, and responsive web-enabled system for centralized student lifecycle management. This system aims to be flexible enough to adapt to evolving policies and administrative rules, streamline processes from institute accreditation to student placement, and integrate with existing NIELIT systems (e.g., online examination, e-learning, e-certification, placement portal). Key goals include providing transparent online accreditation, standardizing course structures, centralizing registration, offering a common student interface for inquiries and enrollment, and generating comprehensive MIS reports for government departments like MeitY and the National Skill Development Agency (NSDA) through its Labour Market Information System (LMIS). Ultimately, the initiative seeks to minimize duplication, paperwork, and manual data transfer, allowing NIELIT officials to focus on core activities and improve data management and retrieval mechanisms.

**Key Methods & Scope:** The project outlines a comprehensive, phased approach:

- Analysis:** Involves preliminary study of existing NIELIT systems, detailed requirement gathering, and finalization of software, database configurations, and data migration strategies.
- Design:** Development of Software Requirements Specification (SRS), Functional Requirement Specification (FRS), high-level and low-level application designs, prototyping, data backup, and Disaster Recovery Management (DRM) policies, ensuring compliance with security guidelines from CERT-In, NIC, and MeitY.
- Development:** Coding and application development, integration with NIELIT's online payment gateway, adherence to web application audit and security features (e.g., OWASP vulnerability-free, strong password policy, session management, audit trails), high availability architecture, and rigorous multi-stage testing (Unit, Integration, System, Functional, Peak Load, UAT).
- Operation & Maintenance:** Identifying and executing training requirements for NIELIT staff, creation of user/technical manuals, and providing post-implementation support, including a one-year warranty and a mandatory offer for a two-year Annual Maintenance Contract (AMC). The system must support at least 1000 concurrent users, be initially bilingual (English and Hindi) with future multilingual provision, and be deployable on the NIC Cloud (Meghraj). The bidder is responsible for a CERT-In security audit and migration of existing and historical data. Functional modules will include Accreditation (defining rules, online applications, scrutiny, inspection, renewal, withdrawal), Expert Management (empowerment, duty assignment, claims processing), Course Management (adding/revising courses, defining rules and dependencies), and Student Information and Registration (unique IDs, online enrollment for all course types, batch uploads, re-registration, transfers, exam scheduling).

**Important Results & Conclusions:** The successful implementation will yield a centralized, highly secure, and efficient platform for managing NIELIT's extensive educational and training ecosystem. It will ensure transparency, reduce administrative overhead, and enable compliance with national reporting standards. A critical outcome is that NIELIT will be the sole proprietor of the developed system's Intellectual Property Rights (IPR) and receive the complete, error-free source code. The project underscores NIELIT's commitment to modernizing its operations through advanced, integrated IT solutions, ensuring robust support and continuous improvement.

Extracted Keywords

15 terms

1 procurement portal 67.4%

2 procurement process 66.6%

3 procurement enroll 64.3%

4 online eprocurement 62.2%

5 uploading procurement 62.0%

6 uploaded procurement 61.7%

7 eprocurement bid 60.8%

8 electronic bids 60.0%

9 nielit provisioning 48.9%

10 procurement activities 48.9%

11 participating procurement 48.8%

12 requirements nielit 48.7%

13 bids nielit 48.4%

14 procedure bidding 47.9%

15 nielit issued 47.6%

30

# Assumptions

## 1. About the RFP Document

- The RFP is well-structured enough for Gemini to extract meaningful metadata (budget, timeline, EMD)
- The document is in English

## 2. About the Company Data

- The company's tech keywords, budget range, team size, and timeline are kept up to date
- The internal data accurately reflects the company's current capacity.

## 3. About the Scoring System

- The weightings used (technical 40%, budget 30%, timeline 30%) fairly represent what matters most in an RFP decision
- The thresholds (ACCEPT  $\geq 70$ , REVIEW  $\geq 50$ , REJECT  $< 50$ ) are reasonable cutoffs
- Technical fit below 40% is always a hard rejection regardless of other scores

## 4. About the Proposal Generation

- The RFP has enough content for RAG retrieval to find meaningful chunks for each proposal section

31

# Hardware and Software Requirements

## Hardware Requirements

Since you're running local embeddings + local LLM (Llama 3.2), hardware matters.

### Minimum Requirements (Basic Testing)

- Processor: Intel i5 (10th Gen) / Ryzen 5
- RAM: 8 GB (16 GB strongly recommended)
- Storage: 20 GB free disk space
- No dedicated GPU required (CPU inference possible but slower)

### Recommended Requirements (Smooth Performance)

- Processor: Intel i7 / Ryzen 7 or better
- RAM: 16–32 GB
- Storage: SSD with 50+ GB free space
- GPU (Optional but Recommended):
  - NVIDIA GPU with 6GB+ VRAM (RTX 2060 or higher)
  - CUDA support for faster LLM inference

32

# Hardware and Software Requirements

## Operating System

- Windows 10/11, Linux (Ubuntu 20.04+ recommended), or macOS
- Linux preferred for better LLM performance and stability

## Backend Technologies

- Python 3.10+
- Django 5.x
- Django Rest Framework (DRF)
- PostgreSQL 14+

## AI / NLP Libraries

- SentenceTransformers (all-MiniLM-L6-v2) – for embeddings
- PyTorch – backend for transformer models
- NumPy / Scikit-learn – similarity & scoring

33

# Hardware and Software Requirements

## Vector Storage

- PostgreSQL with JSONField (custom vector storage)
- Manual cosine similarity implementation

## Local LLM for Proposal Generation

- Ollama
- Llama 3.2 (local model)
- Runs fully offline

## Frontend

- Next.js 14
- React
- Node.js (v18+ recommended)

## Document Processing

- PyPDF2 / pdfplumber (PDF parsing)
- python-docx (DOCX parsing)
- reportlab (PDF export, if used)

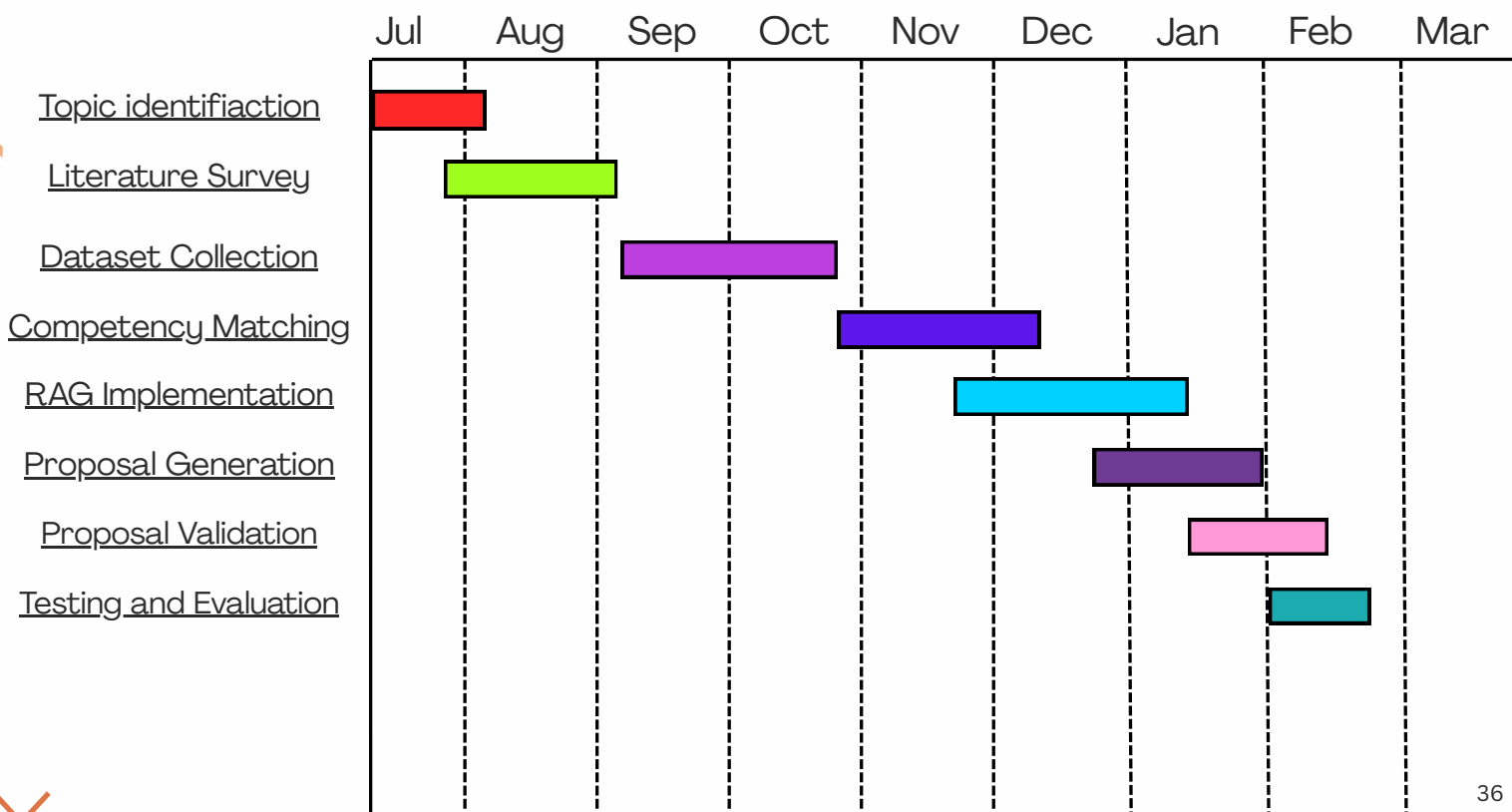
34



# Project Schedule



35



36

## Roles Assigned to Each Member

| Nevin                             | Rahul                             | Niranjana                            | Abidh                         |
|-----------------------------------|-----------------------------------|--------------------------------------|-------------------------------|
| In-house AI creation (extraction) | Competency mapping of the company | In-house AI creation (Summarization) | Compliance verification       |
| Proposal drafting with GenAI      | RFP requirement extraction        | Keyword extraction using In-house AI | Evaluation based on proposals |

37

## Risk and Challenges

- Data Extraction Variability

Budget, timeline, and financial details may not follow consistent patterns, which can affect metadata extraction accuracy and impact the scoring results.

- LLM Hallucination Risk

Although Llama 3.2 is grounded using RAG, there is still a possibility of generating unsupported or slightly fabricated content. Proper prompt design and strong retrieval grounding are required to minimize this risk.

- Data Privacy and Security

RFP documents often contain confidential information. Secure local deployment and proper database access control are necessary to prevent unauthorized access.

38

# Conclusion

This project presents an AI-driven RFP Evaluation and Proposal Generation system that automates RFP analysis, compatibility scoring, and grounded response generation. By integrating semantic retrieval, deterministic scoring, and a locally deployed Llama 3.2 model via Ollama, the system ensures explainability, efficiency, and data privacy through a fully offline architecture. The solution reduces manual effort, improves decision consistency, and demonstrates the practical application of AI in procurement automation.

# References

- [1] P. Nistala, A. Rajbhoj, V. Kulkarni, S. Noronha, and A. Joshi, “An industrial experience report on model-based, AI-enabled proposal development for an RFP/RFI,” [Journal/Conference Name], Elsevier, 2024.
- [2] A. Rajbhoj, P. Nistala, V. Kulkarni, and G. Ganesan, “An RFP system for generating response to a request for proposal,” in Proceedings of the ACM Innovations in Software Engineering Conference (ISEC), Pune, India, 2019, pp. —, doi: 10.1145/3299771.3299779.
- [3] A. Suravarjhula, R. C. Agrawal, S. J. Patel, and R. Gupta, “Retrieval-augmented multi-agent system for rapid statement of work generation,” arXiv preprint arXiv:2508.07569, 2025.  
2023, doi: 10.3390/su15065267.

# References

[4] E. Bilal and O. Åström, “Generative AI for sales: A study on the potential of retrieval augmented generation for response automation in the request for proposal process in telecommunications,” Degree Project, KTH Royal Institute of Technology, Stockholm, Sweden, 2024.

[5] Y. Guo, C. Clavel, M. Kamal Eddine, and M. Vazirgiannis, “Questioning the validity of summarization datasets and improving their factual consistency,” arXiv preprint arXiv:2210.17378, 2022.

[6] A. Smadi et al., “Unlocking the potential of competency exam data with machine learning: Improving higher education evaluation,” Sustainability, vol. 15, no. 6, p. 5267,

# Thank You!



## **Appendix B: Vision, Mission, Programme Outcomes and Course Outcomes**

# **Vision, Mission, Programme Outcomes and Course Outcomes**

## **Institute Vision**

To evolve into a premier technological institution, moulding eminent professionals with creative minds, innovative ideas and sound practical skills, and to shape a future where technology works for the enrichment of mankind.

## **Institute Mission**

To impart state-of-the-art knowledge to individuals in various technological disciplines and to inculcate in them a high degree of social consciousness and human values, thereby enabling them to face the challenges of life with courage and conviction.

## **Department Vision**

To become a centre of excellence in Computer Science and Engineering, moulding professionals catering to the research and professional needs of national and international organizations.

## **Department Mission**

To inspire and nurture students, with up-to-date knowledge in Computer Science and Engineering, ethics, team spirit, leadership abilities, innovation and creativity to come out with solutions meeting societal needs.

## **Programme Outcomes (PO)**

Engineering Graduates will be able to:

**PO1: Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

**PO2: Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

**PO3: Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

**PO4: Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.

**PO5: Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

**PO6: The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

**PO7: Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

**PO8: Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

**PO9: Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

**PO10: Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

**PO11: Life-long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

## Programme Specific Outcomes (PSO)

### PSO1: Computer Science Specific Skills

The ability to identify, analyze and design solutions for complex engineering problems in multidisciplinary areas by understanding the core principles and concepts of computer science and thereby engage in national grand challenges.

### PSO2: Programming and Software Development Skills

The ability to acquire programming efficiency by designing algorithms and applying standard practices in software project development to deliver quality software products meeting the demands of the industry.

### PSO3: Professional Skills

The ability to apply the fundamentals of computer science in competitive research and to develop innovative products to meet the societal needs thereby evolving as an eminent researcher and entrepreneur.

## Course Outcomes (CO)

| SN | CO Description                                                                                                         | Bloom's Taxonomy Level |
|----|------------------------------------------------------------------------------------------------------------------------|------------------------|
| 1  | Model and solve real world problems by applying knowledge across domains.                                              | Apply                  |
| 2  | Develop products, processes or technologies for sustainable and socially relevant application.                         | Apply                  |
| 3  | Function effectively as an individual and as a leader in diverse teams and to comprehend and execute designated tasks. | Apply                  |
| 4  | Plan and execute tasks utilizing available resources within timelines, following ethical and professional norms.       | Apply                  |
| 5  | Identify technology/research gaps and propose innovative/creative solutions.                                           | Analyze                |
| 6  | Organize and communicate technical and scientific findings effectively in written and oral forms.                      | Apply                  |

## Appendix C: CO-PO-PSO Mapping

## CO-PO-PSO Mapping

### MAPPING COURSE OUTCOMES (COs) – PROGRAM OUTCOMES (POs) AND COURSE OUTCOMES (COs) – PROGRAM SPECIFIC OUTCOMES (PSOs)

|     | PO1 | PO2 | PO3 | PO4 | PO5 | PO6 | PO7 | PO8 | PO9 | PO10 | PO11 | PSO1 | PSO2 | PSO3 |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|------|------|------|
| CO1 | 2   | 2   | 2   | 1   |     | 2   | 1   | 1   | 1   | 1    | 1    | 2    | 2    | 2    |
| CO2 | 2   | 2   | 2   |     | 2   | 3   | 2   | 1   | 1   | 1    | 1    | 2    | 2    |      |
| CO3 |     |     |     |     |     |     |     | 3   | 2   | 2    | 1    |      |      | 3    |
| CO4 |     | 1   | 2   |     | 3   | 2   | 3   | 2   | 2   | 3    | 1    |      | 2    | 2    |
| CO5 | 2   | 3   | 3   | 1   | 2   |     |     |     |     | 1    | 3    | 3    |      |      |
| CO6 |     |     | 2   |     |     | 2   | 2   | 2   | 3   | 1    | 1    |      |      | 3    |

Table 1: CO-PO and CO-PSO Mapping

## MAPPING JUSTIFICATION

| Mapping                | Low /<br>Medium /<br>High | Justification                                                                                                     |
|------------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------|
| 101003/CS822U.CO1-PO1  | Medium                    | Applies engineering knowledge, mathematics, and computing fundamentals to model and solve real-world problems.    |
| 101003/CS822U.CO1-PO2  | Medium                    | Requires identification, formulation and analysis of engineering problems before developing solutions.            |
| 101003/CS822U.CO1-PO3  | Medium                    | Involves designing systems or solutions to address identified engineering needs.                                  |
| 101003/CS822U.CO1-PO4  | Low                       | Limited investigation or experimentation may be required to validate the developed models or solutions.           |
| 101003/CS822U.CO1-PO6  | Medium                    | Solutions developed may influence societal, environmental, or sustainability aspects of engineering applications. |
| 101003/CS822U.CO1-PO7  | Medium                    | Encourages consideration of ethical and professional responsibilities while developing solutions.                 |
| 101003/CS822U.CO1-PO8  | Low                       | Some level of teamwork may be required while working on modeling or project tasks.                                |
| 101003/CS822U.CO1-PO9  | Low                       | Requires communication of results and findings through reports or presentations.                                  |
| 101003/CS822U.CO1-PO10 | Low                       | Involves basic planning and execution of tasks while implementing solutions.                                      |
| 101003/CS822U.CO1-PO11 | Low                       | Encourages independent learning and adapting new knowledge to solve real-world problems.                          |
| 101003/CS822U.CO1-PSO1 | Medium                    | Uses core computer science principles to analyze problems and design appropriate technical solutions.             |

| <b>Mapping</b>         | <b>Low /<br/>Medium /<br/>High</b> | <b>Justification</b>                                                                                     |
|------------------------|------------------------------------|----------------------------------------------------------------------------------------------------------|
| 101003/CS822U.CO1-PSO2 | Medium                             | Involves algorithm design and application of software development practices to implement solutions.      |
| 101003/CS822U.CO1-PSO3 | Medium                             | Applies computer science fundamentals in research or innovative problem solving for societal needs.      |
| 101003/CS822U.CO2-PO1  | Medium                             | Utilizes engineering knowledge to design and develop products or systems.                                |
| 101003/CS822U.CO2-PO2  | Medium                             | Requires analysis of user needs and technological feasibility before development.                        |
| 101003/CS822U.CO2-PO3  | Medium                             | Focuses on design and development of solutions addressing real-world requirements.                       |
| 101003/CS822U.CO2-PO5  | Medium                             | Requires use of modern engineering and IT tools for system or product development.                       |
| 101003/CS822U.CO2-PO6  | High                               | Emphasizes sustainability and societal impact during solution development.                               |
| 101003/CS822U.CO2-PO7  | Medium                             | Ethical and professional responsibilities are considered when developing socially relevant technologies. |
| 101003/CS822U.CO2-PO8  | Low                                | Development activities may require collaboration among team members.                                     |
| 101003/CS822U.CO2-PO9  | Low                                | Involves communicating design ideas and development outcomes.                                            |
| 101003/CS822U.CO2-PO10 | Low                                | Requires basic planning and coordination during development tasks.                                       |
| 101003/CS822U.CO2-PO11 | Low                                | Encourages continuous learning of emerging technologies while building solutions.                        |
| 101003/CS822U.CO2-PSO1 | Medium                             | Applies computer science fundamentals to design technically feasible solutions.                          |
| 101003/CS822U.CO2-PSO2 | Medium                             | Uses programming and software development practices to implement solutions.                              |



| <b>Mapping</b>         | <b>Low /<br/>Medium /<br/>High</b> | <b>Justification</b>                                                                          |
|------------------------|------------------------------------|-----------------------------------------------------------------------------------------------|
| 101003/CS822U.CO3-PO8  | High                               | Strongly relates to functioning effectively as a member or leader in multidisciplinary teams. |
| 101003/CS822U.CO3-PO9  | Medium                             | Requires effective communication among team members during collaboration.                     |
| 101003/CS822U.CO3-PO10 | Medium                             | Team leadership involves project coordination and task management.                            |
| 101003/CS822U.CO3-PO11 | Low                                | Team interaction supports learning from peers and adapting new approaches.                    |
| 101003/CS822U.CO3-PSO3 | High                               | Professional collaboration and teamwork support innovative computer science solutions.        |
| 101003/CS822U.CO4-PO2  | Low                                | Requires analysis and planning before executing project tasks.                                |
| 101003/CS822U.CO4-PO3  | Medium                             | Involves designing and implementing project solutions based on planned activities.            |
| 101003/CS822U.CO4-PO5  | High                               | Requires effective use of modern tools and resources during project execution.                |
| 101003/CS822U.CO4-PO6  | Medium                             | Considers societal and environmental impact while implementing solutions.                     |
| 101003/CS822U.CO4-PO7  | High                               | Emphasizes adherence to ethical practices and professional responsibilities.                  |
| 101003/CS822U.CO4-PO8  | Medium                             | Task execution may involve teamwork and coordination among team members.                      |
| 101003/CS822U.CO4-PO9  | Medium                             | Requires communication of progress, outcomes and documentation.                               |
| 101003/CS822U.CO4-PO10 | High                               | Focuses on project management including scheduling, resource allocation and task execution.   |
| 101003/CS822U.CO4-PO11 | Low                                | Encourages adaptability and continuous learning during project implementation.                |

| <b>Mapping</b>         | <b>Low /<br/>Medium /<br/>High</b> | <b>Justification</b>                                                                    |
|------------------------|------------------------------------|-----------------------------------------------------------------------------------------|
| 101003/CS822U.CO4-PSO2 | Medium                             | Uses software development practices to execute and manage project tasks.                |
| 101003/CS822U.CO4-PSO3 | Medium                             | Applies computer science knowledge in practical project development environments.       |
| 101003/CS822U.CO5-PO1  | Medium                             | Uses engineering knowledge to understand technological limitations and opportunities.   |
| 101003/CS822U.CO5-PO2  | High                               | Requires deep problem analysis and identification of research gaps.                     |
| 101003/CS822U.CO5-PO3  | High                               | Proposes innovative design solutions to address identified gaps.                        |
| 101003/CS822U.CO5-PO4  | Low                                | May involve limited investigation or validation of proposed ideas.                      |
| 101003/CS822U.CO5-PO5  | Medium                             | Uses appropriate tools and techniques to analyze and develop innovative solutions.      |
| 101003/CS822U.CO5-PO10 | Low                                | Requires minimal planning while proposing new solution approaches.                      |
| 101003/CS822U.CO5-PO11 | High                               | Strongly promotes independent learning and exploration of emerging technologies.        |
| 101003/CS822U.CO5-PSO1 | High                               | Applies computer science fundamentals to identify and solve complex technical problems. |
| 101003/CS822U.CO6-PO3  | Medium                             | Requires presentation of design concepts and developed solutions.                       |
| 101003/CS822U.CO6-PO6  | Medium                             | Communication may include discussion of societal or environmental implications.         |
| 101003/CS822U.CO6-PO7  | Medium                             | Requires responsible and ethical communication of technical results.                    |
| 101003/CS822U.CO6-PO8  | Medium                             | Communication supports collaboration within team environments.                          |
| 101003/CS822U.CO6-PO9  | High                               | Strongly relates to effective written and oral communication of technical findings.     |

| <b>Mapping</b>         | <b>Low /<br/>Medium /<br/>High</b> | <b>Justification</b>                                                            |
|------------------------|------------------------------------|---------------------------------------------------------------------------------|
| 101003/CS822U.CO6-PO10 | Low                                | Requires structured reporting and documentation in project management contexts. |
| 101003/CS822U.CO6-PO11 | Low                                | Encourages continuous improvement in communication and presentation skills.     |
| 101003/CS822U.CO6-PSO3 | High                               | Communication of innovative computer science solutions and research outcomes.   |